EKNATHALABS · INTERVIEW PREP

Observability & Monitoring — EknathaLabs Interview Guide

Platform, SRE & DevOps interview guide

Questions	**80 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/observability_interview_guide.html**
Printed	**2026-10-21**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

certs.eknathalabs.com

Q1 What is the difference between metrics, logs, and traces — the three pillars of observability?

Metrics

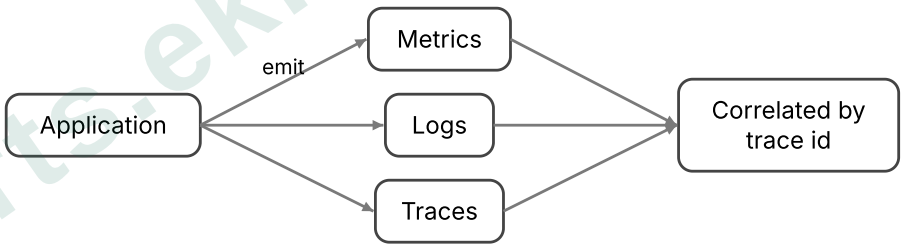
🎙 Spoken answer · ~84 s

Metrics, logs, and traces are three complementary telemetry types, each answering a different question. Metrics are numeric measurements aggregated over time, like request rate, error rate, or CPU usage. They are cheap to store, fast to query, and ideal for dashboards and alerting, but they lose per-request detail. Logs are timestamped records of discrete events, giving rich context such as error messages and stack traces. They're essential for understanding exactly what happened, but are voluminous and expensive to store and search. Traces follow a single request as it moves across services, breaking it into spans with timing, so you can see where latency or failure occurs in a distributed system. In practice I use them together: an alert fires from a metric, telling me something is wrong; a trace shows which service or call is slow; and logs for that span or trace ID give the detailed reason. Correlation is the key, so I embed trace IDs in logs and attach exemplars to metrics. Observability is the ability to ask new questions about system behaviour from its outputs, without shipping new code, so high-cardinality, well-structured data matters. I'd start with metrics and logs, then add tracing as architecture becomes more distributed.

Likely follow-ups

- How do you correlate the three signals during an incident?
- Which signal would you invest in first?

🗺 Three signals



Each pillar answers a different question about system health.

```
Metrics  — What is happening? (quantitative time-series, cheap, fast to query)
```

```
Logs     — Why is it happening? (discrete events, full context per event)
```

```
Traces   — Where is it happening? (request path across services, latency per hop)
```

Metrics alert you, logs explain the event, traces show blast radius. Use all three: a latency spike in metrics leads you to filter logs by trace_id, then view the trace waterfall to pinpoint the slow service.

Cardinality: Keep metric label values low-cardinality (status_code, region, pod_name — not user_id). High-cardinality labels create millions of series and crash Prometheus. Use logs for high-cardinality data.

certs.eknathalabs.com

📚 **Official docs**

- OpenTelemetry: Signals (traces, metrics, logs)

Q2 What are the four golden signals — how do you use them to monitor a production service?

Metrics

🎙 **Spoken answer · ~78 s**

The four golden signals, from Google's SRE book, are latency, traffic, errors, and saturation. Latency is how long requests take, and you should separate successful from failed requests, since fast errors can hide a problem; track percentiles like p95 and p99, not averages. Traffic is demand on the system, such as requests per second or sessions. Errors are the rate of failing requests, including explicit failures like HTTP 5xx, implicit ones like a 200 with wrong content, and policy failures like responses slower than a threshold. Saturation is how full the service is, such as CPU, memory, queue depth, or connection pool usage, and it's often the leading indicator of trouble. For every user-facing service I build one dashboard row for each signal, and base alerts on symptoms users feel, mainly errors and latency, with saturation alerts to warn before the limit. Use RED, rate, errors, duration, for request-driven services, and USE, utilisation, saturation, errors, for resources. Golden signals give a consistent starting point that works for any service, make onboarding easy, and tie directly to SLOs: the SLIs are typically derived from latency and errors.

Likely follow-ups

- How does RED differ from USE?
- Why measure latency percentiles rather than averages?

```
Latency     — time to serve a request (use P99, never average)
```

```
Traffic     — requests/second, messages/second
```

```
Errors      — rate of failed requests (5xx, timeouts)
```

```
Saturation  — how full? (CPU %, queue depth, connection pool %)
```

```
# PromQL:
```

```
rate(http_requests_total{code=~"5.."}[5m]) / rate(http_requests_total[5m])  # error rate
```

```
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))     # P99
```

Always use percentiles for latency — averages hide tail latency. If you can only have one dashboard per service, make it the golden signals dashboard.

Start here: Instrument every service with golden signals before any custom metrics. If P99 latency is fine, error

certs.eknathalabs.com

rate is zero, and saturation is low — the service is healthy regardless of internal details.

📚 **Official docs**

- Google SRE Book: Monitoring Distributed Systems

Q3 What is Prometheus — how does pull-based metrics collection work?

Metrics

🎙 **Spoken answer · ~83 s**

Prometheus is an open-source monitoring system with a time-series database, a query language called PromQL, and alerting. It uses a pull model: the Prometheus server periodically scrapes HTTP endpoints, usually /metrics, exposed by applications and exporters, and stores samples with timestamps and labels. Targets are found through static configuration or service discovery, such as Kubernetes, Consul, or cloud APIs, and relabelling controls which targets are scraped and how they're labelled. Pull has advantages: Prometheus controls the scrape rate, a failed scrape directly signals that a target is down through the up metric, you can query targets manually with curl, and there's no need for each application to know where to send data. For short-lived jobs that can't be scraped, the Pushgateway is available, though it should be used sparingly. Metric types are counter, gauge, histogram, and summary. Data is stored locally, so for long-term retention and high availability you add Thanos, Cortex, Mimir, or remote write to a managed service. Alerting rules evaluate PromQL and send to Alertmanager. Be careful with high-cardinality labels such as user IDs, since each label combination creates a new series and can exhaust memory. In Kubernetes, the Prometheus Operator and ServiceMonitors make this declarative.

Likely follow-ups

- Why is high label cardinality a problem?
- When would you use the Pushgateway?

```
prometheus --config.file=prometheus.yml
```

```
curl http://myapp:8080/metrics             # app exposes /metrics endpoint
```

```
scrape_configs:
```

```
  - job_name: myapp
```

```
    kubernetes_sd_configs: [{role: pod}]   # auto-discover K8s pods
```

```
    relabel_configs: [...]                 # filter by annotation
```

```
promtool check config prometheus.yml       # validate config
```

certs.eknathalabs.com

Pull model: Prometheus controls scrape rate, discovers targets via service discovery, detects targets that disappear (up metric = 0). In Kubernetes use ServiceMonitor / PodMonitor CRDs (Prometheus Operator) for GitOps-based scrape config — no central prometheus.yml editing per service.

Long-term storage: Default retention 15 days. Use remote_write to Thanos, Cortex, or GCP Managed Prometheus for unlimited retention and multi-cluster global query.

📚 **Official docs**

- Prometheus: When to use the Pushgateway
- Prometheus: Overview

Q4 How do you write effective PromQL queries — what are the key functions?

Metrics

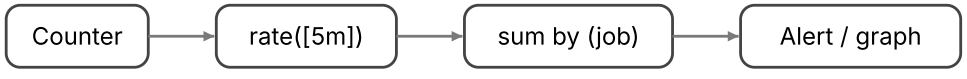
🎙 **Spoken answer · ~87 s**

PromQL is a functional query language for time series. The basics: an instant vector selector like http_requests_total{job="api"}, filtered with label matchers, and a range vector like that with [5m], which gives samples over a window. Key functions: rate() computes per-second average increase of a counter over a window and handles counter resets; increase() gives total increase; irate() uses the last two samples and suits volatile graphs; sum, avg, max and others aggregate, with by or without to choose which labels to keep; histogram_quantile() estimates percentiles from histogram buckets, normally applied to rate of the bucket series summed by le; and topk picks the biggest series. Other useful ones: predict_linear for forecasting disk fill, delta and deriv for gauges, absent to alert when a series is missing, and offset to compare against a past period. Vector matching with on and ignoring joins series from different metrics, for example dividing errors by total requests. Practices: always apply rate to counters before aggregating, never sum counters directly; pick a window of at least four times the scrape interval; use recording rules to precompute expensive queries; and keep label cardinality under control. A typical error ratio is the rate of 5xx requests divided by the rate of all requests, each summed by service.

Likely follow-ups

- Why apply rate before sum on counters?
- How does histogram_quantile work?

🗺 **PromQL pipeline**




```
rate(metric[5m])                                # per-second rate (use for counters)
```

```
histogram_quantile(0.99, rate(duration_bucket[5m])) # P99 from histogram
```

```
sum by (pod) (rate(metric[5m]))                 # aggregate by label
```

certs.EknathaLabs.com

```
topk(5, metric)                                    # top 5 values
```

```
absent(up{job="myapp"})                            # alert when target disappears
```

```
predict_linear(disk_used[1h], 3600*24)             # predict disk full in 24h
```

```
# Error rate ratio:
```

```
sum(rate(requests_total{code=~"5.."}[5m])) / sum(rate(requests_total[5m]))
```

rate() vs irate(): Use rate() for alerts and dashboards (smoothed). Use irate() only for detecting very brief spikes. irate() uses only the last 2 data points — slow scrape intervals make it unreliable.

📚 **Official docs**

- Prometheus: Querying basics (PromQL)

Q5 What is a histogram vs summary in Prometheus — when do you use each?

Metrics

🎙 **Spoken answer · ~84 s**

Both histograms and summaries record observations like request durations, but they work differently. A histogram counts observations in configurable buckets, such as less than 100 ms, less than 250 ms, and so on, and exposes cumulative bucket counters plus a sum and count. Quantiles are computed at query time with histogram_quantile, from the buckets, which makes them aggregatable across many instances: you can sum the buckets across all pods and compute a fleet-wide p99. The trade-offs are that accuracy depends on bucket boundaries, so you must choose buckets around your SLO thresholds, and the result is an estimate. A summary calculates quantiles on the client side, in the application, over a sliding time window, and exposes them directly. They can be more accurate for the configured quantiles, but you cannot aggregate them across instances, since averaging percentiles is mathematically meaningless, and you can't pick other quantiles later. For these reasons, I use histograms by default in distributed services, especially because SLOs need fleet-wide latency, and the bucket counts also let me compute the proportion of requests faster than a threshold directly. Use summaries only for a single instance or when exact quantiles matter. Native histograms in newer Prometheus reduce the bucket-choice problem.

Likely follow-ups

- Why can't you average summary quantiles across pods?
- How do you choose histogram buckets?

```
# Histogram: buckets defined upfront, percentile computed at query time
```

```
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))
```

certs.eknathalabs.com

```
# Summary: percentile computed in client, not aggregatable across instances
```

```
http_request_duration_seconds{quantile="0.99"} 0.42
```

Histograms: flexible (any percentile at query time), aggregatable across replicas (add buckets from N pods). Summaries: accurate client-side, but cannot be aggregated — useless for multi-replica services. Always use histograms for services with multiple replicas. Choose bucket boundaries around your SLO (if SLO is P99 under 500ms, add buckets at 0.1, 0.25, 0.5, 0.75, 1.0).

📚 **Official docs**

- Prometheus: Histograms and summaries
- Prometheus: Overview

Q6 How does structured logging work — why is it better than plain text logs? Logging

🎙 **Spoken answer · ~83 s**

Structured logging means emitting logs as machine-parseable records, usually JSON, with consistent named fields rather than free-form text sentences. A line such as a timestamp, level, service, message, trace ID, user ID, and request path becomes a set of key-value pairs. Benefits: queries become precise and fast, for example filtering by status code or user without fragile regular expressions; fields can be indexed, aggregated, and graphed; parsing is cheap and doesn't break when someone rewrites a message; and logs can be correlated across services using shared fields like trace ID and request ID. It also supports consistent schemas across teams, which makes dashboards and alerts reusable. Practices: use a standard logging library with a JSON formatter; pick a common set of field names, such as the OpenTelemetry semantic conventions; include context like service name, version, environment, and trace and span IDs; use levels meaningfully; keep the message constant and put variable data in fields; avoid logging sensitive data like passwords, tokens, or personal information; avoid high-cardinality surprises; and write to stdout in containers, letting the platform collect them. Be mindful of volume and cost, and sample noisy debug logs. Structured logs turn logging from text to searchable data.

Likely follow-ups

- What fields should every log line carry?
- Why keep the log message constant and put variables in fields?

```
# Unstructured (bad): 2024-01-15 ERROR payment failed for user 12345
```

```
# Structured JSON (queryable):
```

```
{"ts":"2024-01-
15T10:23:01Z","level":"error","event":"payment_failed","user_id":12345,"amount":99.99,"trac
e_id":"abc123"}
```

```
# Query in Cloud Logging:
```

certs.eknathalabs.com

```
jsonPayload.event="payment_failed" AND jsonPayload.amount > 100
```

```
# Query in Loki:
```

```
{app="payments"} | json | event="payment_failed" | amount > 100
```

Structured logging enables filtering by any field, aggregation (count failures by error type), and correlation (join on trace_id). Always include: timestamp (ISO 8601), level, service name, trace_id/span_id (links to distributed trace), request_id. Never log passwords, tokens, or PII at any level.

📚 **Official docs**

- OpenTelemetry: Logs

Q7 How do you build a log aggregation pipeline — ELK vs Loki vs Cloud Logging?

Logging

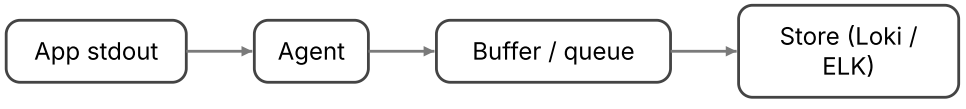
🎙 **Spoken answer · ~88 s**

A log aggregation pipeline has collection, transport and processing, storage, and query layers. Collection: agents such as Fluent Bit, Vector, Promtail, or the OpenTelemetry Collector run on each node as a DaemonSet in Kubernetes, reading container stdout. Processing: parse, enrich with Kubernetes metadata, filter, and redact sensitive fields, optionally via a buffer like Kafka to absorb bursts. Storage and query differ by stack. ELK or the Elastic stack indexes the full text of every log in Elasticsearch, giving powerful search and analytics with Kibana, but it requires significant compute and storage, plus operational effort for sizing, shards, and lifecycle management. Loki indexes only labels, such as namespace and app, and stores compressed log chunks in cheap object storage; queries use LogQL and filter content at query time. It's far cheaper and simpler and integrates with Grafana and Prometheus label conventions, but full-text search over large ranges is slower. Cloud Logging on GCP, or CloudWatch on AWS, is managed: no infrastructure to run, integrated with the platform and IAM, but costs scale with volume and it can be less flexible. My choice: Loki for Kubernetes teams who want low cost and tight Grafana integration; Elastic where heavy full-text analytics or security use cases matter; managed services for small teams who want minimal operations.

Likely follow-ups

- Why is Loki cheaper than Elasticsearch?
- What does a buffer like Kafka add to the pipeline?

🗺 **Log pipeline**




```
# ELK: Fluent Bit → Elasticsearch → Kibana (full-text, expensive)
```

corrs.eknathalabs.com

```
# Loki: Promtail → Loki → Grafana (label-indexed, cheap, PromQL-like)
```

```
# GCP: container stdout → Cloud Logging (zero-config for GKE)
```

```
[FILTER]
```

```
    Name   kubernetes
```

```
    Merge_Log On       # parse JSON logs into structured fields
```

Loki indexes only labels (not content) — typically 10-20x cheaper storage than Elasticsearch. For teams using Grafana+Prometheus, Loki is a natural fit. GCP Cloud Logging is zero-ops for GKE but vendor-locked. ELK is better for complex full-text search and compliance requirements. Cost at scale is the key driver.

📚 **Official docs**

- Cloud Logging
- Google SRE Book: Distributed tracing & logs (Monitoring chapter)
- Grafana Loki documentation

Q8 Log volume is exploding and costs are too high — how do you reduce it without losing signal? Logging

🎙 **Spoken answer · ~88 s**

Reducing log cost without losing signal means being deliberate about what's collected, how long it's kept, and where. First, find the sources: measure volume by service, level, and message pattern, because usually a few noisy sources produce most of the data. Then act. Remove or lower noise: turn off debug logging in production, delete repetitive health check and access logs for successful probes, and fix chatty loops. Sample high-volume, low-value logs, such as keeping one in ten successful requests while keeping all errors. Filter at the agent or collector, before ingestion, as that is where savings are greatest. Trim payloads: drop unneeded fields, avoid logging whole request bodies, and shorten stack traces where possible. Use tiered retention: keep recent logs hot and searchable for a week or two, archive to cheap object storage for compliance, and delete the rest; set different retention by criticality. Convert logs to metrics where you only need counts and rates, such as error counters, and drop the raw lines. Choose cheaper storage like Loki over full indexing. Set per-team quotas and budgets with visibility, so teams own their cost. Protect the signal: never sample errors, security events, or audit logs, and rely on traces and metrics for broad visibility. Review volume regularly, as it drifts upward.

Likely follow-ups

- What would you never sample or drop?
- How can converting logs to metrics save cost?

certs.eknathalabs.com

```
# Fluent Bit: drop below WARNING for noisy services
```

```
[FILTER]
```

```
    Name   grep
```

```
    Exclude log level debug|info
```

```
# Cloud Logging exclusion filter:
```

```
resource.type="k8s_container" severity < WARNING
```

```
# Loki per-tenant rate limiting:
```

```
ingestion_rate_mb: 10
```

Strategies: (1) Level filtering — keep WARN+ always, sample INFO, drop DEBUG. (2) Head sampling — drop N% of healthy-looking lines at the agent. (3) Aggregation — replace N log lines with a single metric. (4) Retention tiering — hot (7 days), cold (90 days), archive (1 year). Reducing volume 80% while keeping 100% of error signal is achievable with level filtering alone.

Never drop ERROR logs. Only reduce INFO and DEBUG. Error logs are the signal you need most during incidents.

📚 **Official docs**

- OpenTelemetry: Logs

certs.eknathalabs.com

Q9 How does distributed tracing work — what is a trace, span, and context propagation?

Tracing

🎙 Spoken answer · ~88 s

Distributed tracing follows a single request across multiple services. A trace is the complete journey of one request and is identified by a trace ID. It's made of spans, each representing a unit of work, like an HTTP handler, a database query, or a message publish, with a span ID, parent span ID, start time, duration, status, and attributes. Spans form a tree showing causal relationships and where time is spent. Context propagation is what links spans across process boundaries: when a service calls another, it injects the trace context, trace ID, parent span ID, and sampling flag, into outgoing headers, such as the W3C traceparent header, or message metadata; the receiving service extracts it and creates child spans under the same trace. Without propagation, traces break into fragments. Instrumentation can be automatic through agents and libraries or manual for business logic. Spans are exported to a backend, such as Jaeger, Tempo, or Cloud Trace, usually through the OpenTelemetry Collector. Because tracing every request is costly, sampling is used: head-based, deciding at the start, or tail-based, deciding after seeing the whole trace, so errors and slow traces are always kept. Traces reveal latency bottlenecks, dependencies, and errors that metrics can't localise, and trace IDs in logs tie it all together.

Likely follow-ups

- What is the difference between head-based and tail-based sampling?
- What happens if one service doesn't propagate context?

```
Trace  — the entire journey of one request across all services
```

```
Span   — one unit of work (HTTP handler, DB query, RPC call)
```

```
Parent — span knows its parent (forms a tree)
```

```
# W3C traceparent header propagated between services:
```

```
traceparent: 00-{traceId}-{spanId}-{flags}
```

```
# Each service reads header, creates child span, passes downstream
```

Context propagation is critical: when service A calls B, A injects trace context into HTTP headers. B extracts it and creates a child span with the same trace ID — forming a connected tree. OpenTelemetry handles propagation automatically. Without it, traces are disconnected single-service islands.

Sampling: Head-based (decide at root, simple), tail-based (buffer + keep errors, smarter but expensive). GCP Cloud Trace uses tail-based sampling automatically.

certs.eknathalabs.com

📚 **Official docs**

- OpenTelemetry: Signals (traces, metrics, logs)

Q10 How do you implement OpenTelemetry in a production Kubernetes environment?

Tracing

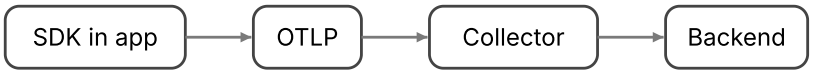
🎙 **Spoken answer · ~81 s**

On Kubernetes, I deploy OpenTelemetry as a vendor-neutral pipeline. Applications are instrumented with the OTel SDKs or auto-instrumentation, which the OpenTelemetry Operator can inject through annotations on pods for Java, Python, Node.js, and .NET without code changes. They send OTLP data to a collector. A common architecture has two tiers: a collector running as a DaemonSet agent on each node, collecting local telemetry plus host and Kubernetes metadata; and a central collector Deployment as a gateway, handling heavy processing, such as batching, tail-based sampling, filtering, redaction, and routing. Collector configuration is a pipeline of receivers, processors, and exporters; use the memory_limiter and batch processors to protect stability, and the k8sattributes processor to enrich with pod, namespace, and deployment labels. Exporters send traces to Tempo or Jaeger, metrics to Prometheus or a remote write endpoint, and logs to Loki or another backend; you can switch backends without changing application code. Set resource attributes like service name, version, and environment consistently. For production: size and autoscale the collectors, enable retries and queues, secure OTLP with TLS, monitor the collectors themselves, control sampling and cardinality for cost, and roll out gradually, starting with the most critical services.

Likely follow-ups

- Why use both an agent and a gateway collector?
- What does the k8sattributes processor add?

🗺 **OpenTelemetry**




```
helm install otel-collector open-telemetry/opentelemetry-collector -f values.yaml
```

```
# Configure via env vars in pod spec:
```

```
OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector.monitoring:4317
```

```
OTEL_SERVICE_NAME=payments-service
```

```
OTEL_TRACES_SAMPLER=parentbased_traceidratio
```

```
OTEL_TRACES_SAMPLER_ARG=0.1    # sample 10%
```

certs.eknathalabs.com

```
# Auto-instrumentation via OTel Operator annotation:
```

```
kubectl annotate pod mypod instrumentation.opentelemetry.io/inject-python=true
```

The OTel Collector acts as a local proxy — apps send spans to it, it forwards to Jaeger/Tempo/Cloud Trace. Decouples application from backend. The OTel Operator auto-instruments pods via annotation (Java, Python, Node.js, .NET, Go) with zero code changes and no Dockerfile modification.

📚 **Official docs**

- OpenTelemetry documentation

Q11 What makes a good alert — what is alert fatigue and how do you fight it?

Alerting

🎙 **Spoken answer · ~85 s**

A good alert is actionable, urgent, and tied to user impact: when it fires, a human needs to do something now, and they know what. Principles: alert on symptoms, such as high error rate or latency affecting users, rather than on every cause like high CPU; include context in the alert, such as the service, severity, a dashboard link, and a runbook; avoid duplicates by grouping; and use severity levels so only urgent things page, with lower severity going to tickets or chat. Alert fatigue happens when people receive too many noisy, non-actionable, or duplicated alerts, so they start ignoring them, and real incidents are missed. Fighting it: audit alerts regularly and delete or fix those that are never actioned; track metrics such as pages per on-call shift and the percentage that required action; tune thresholds and add a for duration to avoid flapping on brief spikes; use SLO burn-rate alerts rather than fixed thresholds; group and deduplicate related alerts, with inhibition so a root-cause alert suppresses downstream ones; route by ownership; and make every page a learning opportunity, with postmortem follow-ups to prevent repeats. Treat on-call health as a first-class goal. If an alert can wait until morning, it shouldn't page at night.

Likely follow-ups

- Why alert on symptoms rather than causes?
- What metrics tell you your alerts are unhealthy?

certs.eknathalabs.com

```
Good alert: actionable + urgent + accurate + contextual
```

```
Bad alert: "CPU > 80%" fires constantly with no action needed
```

```
# Every alert must have:
```

```
summary: "Payment service error rate above SLO"
```

```
description: "Error rate 2.3%, SLO threshold 0.1%, ~1200 users/min affected"
```

```
runbook_url: "https://wiki/runbooks/payment-errors"
```

```
dashboard_url: "https://grafana/d/payments?from=now-1h"
```

Alert fatigue: engineers silence alerts without investigating, become desensitised, miss real incidents. Fix: delete alerts that never need action, raise noisy thresholds, convert informational alerts to dashboard panels, require every alert to have a runbook. If you cannot describe in one sentence what action to take, the alert should not exist.

Symptom-based alerting: Alert on user-visible symptoms (error rate, latency) not internal causes (CPU%). High CPU that does not affect users is a dashboard item, not an alert.

📚 **Official docs**

- Google SRE Book: Practical Alerting

certs.eknathalabs.com

Q12 How does Alertmanager work — routing, grouping, inhibition, and silences?

Alerting

🎙 Spoken answer · ~85 s

Alertmanager receives alerts from Prometheus and handles their notification. Its main functions: grouping, which batches related alerts, for example all alerts from one cluster or service, into a single notification, using group_by labels, so an outage doesn't send hundreds of messages; deduplication of identical alerts coming from multiple Prometheus replicas; routing, via a tree of routes matching on labels, such as severity or team, with each route sending to a receiver like Slack, PagerDuty, or email; inhibition, where a firing alert suppresses others, like a cluster-down alert silencing all the individual service alerts caused by it; and silences, temporary mutes created manually through the UI or API, matching labels for a set duration, such as during planned maintenance. Timing parameters control behaviour: group_wait is how long to wait to collect the initial group, group_interval is the delay before sending updates about a group, and repeat_interval controls how often an unresolved alert is re-sent. For high availability, run several Alertmanager instances in a gossip cluster, with each Prometheus sending to all of them. Keep configuration in version control, use templates for useful messages, and test routing with amtool. Also monitor Alertmanager itself, including a deadman switch alert that always fires to prove the pipeline works.

Likely follow-ups

- What is a deadman switch alert?
- How does inhibition differ from a silence?

```
route:
  group_by: [alertname, cluster, service]
  group_wait: 30s
  routes:
  - match: {severity: critical} → receiver: pagerduty
  - match: {team: platform}    → receiver: slack-platform
inhibit_rules:                  # suppress child when parent fires
  - source_match: {alertname: ClusterDown}
    target_match: {severity: warning}
    equal: [cluster]
```

```
amtool --alertmanager.url=http://localhost:9093 config routes test --
verify.receivers=pagerduty alertname=HighErrorRate
```

certs.eknathalabs.com

Grouping batches related alerts into one notification (10 pod failures → 1 notification). Inhibition suppresses lower-severity alerts when a higher one fires — if the cluster is down, no point getting individual pod alerts. Silences mute during maintenance windows. Test routing with amtool in CI before applying config.

📚 **Official docs**

- Prometheus: Alertmanager

Q13 What is an SLI, SLO, and SLA — how do you define and measure them?

SLOs

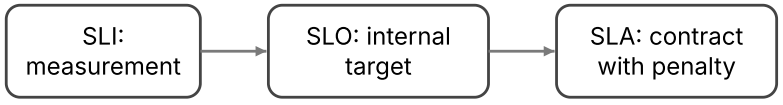
🎙 **Spoken answer · ~86 s**

These three terms form a chain. An SLI, service level indicator, is a quantitative measure of some aspect of service behaviour, usually expressed as a ratio of good events to valid events, like the proportion of requests that succeed or complete under 300 ms. An SLO, service level objective, is the target for that SLI over a time window, for example 99.9 percent of requests succeed over 30 days. It's an internal goal that guides engineering decisions. An SLA, service level agreement, is a contract with customers that specifies consequences, usually financial credits, if targets are missed; SLAs should be looser than SLOs so you have a safety margin. To define them: start from user journeys, pick a few SLIs that reflect user happiness, typically availability and latency, measured as close to the user as practical, such as at the load balancer or from synthetic probes; choose targets from historical data and business needs, not perfection, since 100 percent is neither achievable nor affordable; pick a window, rolling thirty days is common. Measure with Prometheus metrics or cloud monitoring, and show on a dashboard. SLOs create a shared language between product and engineering and enable error budgets. Review and revise them as the service and expectations change.

Likely follow-ups

- Why should an SLA be looser than the SLO?
- What makes a good SLI?

🗺 **SLI, SLO, SLA**




```
SLI — the metric: fraction of successful requests, P99 latency
```

```
SLO — the target: success rate > 99.9% over 30 days
```

```
SLA — the contract: SLO + consequences for breach
```

```
# PromQL availability SLI:
```

```
1 - (sum(rate(requests_total{code=~"5.."}[5m])) / sum(rate(requests_total[5m])))
```

certs.eknathalabs.com

SLIs must be user-facing — not CPU utilisation. Bad SLI: CPU %. Good SLI: request success rate as seen by users. Start with your measured baseline + small improvement target. Rolling 30-day windows are better than calendar windows — always reflect the most recent 30 days of behaviour.

📚 **Official docs**

- Google SRE Book: Service Level Objectives

Q14 What is an error budget — how does it change how you run on-call?

SLOs

🎙 **Spoken answer · ~85 s**

An error budget is the amount of unreliability your SLO allows: one minus the target. A 99.9 percent SLO over 30 days permits 0.1 percent failures, about 43 minutes of downtime or the equivalent proportion of failed requests. You track how much of that budget has been consumed. The budget turns reliability from a vague argument into a shared, quantitative decision tool. When plenty of budget remains, the team can take risks: ship faster, run experiments, and do riskier migrations. When the budget is nearly exhausted, you slow down: freeze non-essential releases, prioritise reliability work, and investigate causes. That aligns incentives, because developers want to ship and SREs want stability, and the budget arbitrates objectively. For on-call, it changes behaviour: instead of paging on every blip, you page when the budget is burning fast enough to be at risk, using burn-rate alerts. Slow, small burns generate tickets rather than pages. It also informs how much toil and incident investment is justified, and provides data for postmortems and prioritisation. Agree on an error budget policy in advance, signed by product and engineering, stating what happens when the budget is spent, so decisions aren't negotiated during a crisis. Review consumption regularly, and exclude agreed maintenance carefully.

Likely follow-ups

- What does an error budget policy contain?
- How does a burn rate relate to the budget?

```
SLO: 99.9% over 30 days = 0.1% budget = 43.8 min downtime allowed
```

```
# Error budget remaining:
```

```
1 - (observed_error_rate / allowed_error_rate)
```

```
50% budget left mid-month = healthy, deploy freely
```

```
5% budget left on day 10 = freeze deployments, fix reliability
```

Error budget removes the subjective argument about whether to deploy. When budget is healthy, deploy freely. When nearly exhausted, freeze deployments until it recovers. This creates natural alignment: both product and engineering teams share the same budget. The policy should be written explicitly: "if error

certs.eknathalabs.com

budget drops below 10%, all non-reliability deployments stop."

📚 **Official docs**

- Google SRE Book: Embracing Risk (error budgets)
- Google SRE Workbook: On-Call

Q15 How do you implement SLO-based alerting — burn rate alerts vs threshold alerts?

SLOs

🎙 **Spoken answer · ~78 s**

Burn-rate alerting pages you based on how quickly the error budget is being consumed, rather than on a raw error threshold. Burn rate is the observed error rate divided by the allowed rate: a burn rate of one uses exactly the whole budget by the end of the window, while ten uses it in a tenth of the time. Simple threshold alerts, like error rate above one percent for five minutes, have problems: they're either too sensitive to tiny blips or too slow to catch real, slow burns, and they don't relate to the SLO. The recommended approach is multi-window, multi-burn-rate alerting. For a 30-day SLO, a fast alert pages when the burn rate is about 14 over both one hour and five minutes, meaning two percent of the budget is gone in an hour; a slower page at a burn rate of 6 over six hours and thirty minutes; and a ticket at a burn rate of 1 over three days. Requiring both a long and a short window means the alert fires quickly but also resets quickly once the problem is fixed, reducing false positives.

Likely follow-ups

- Why use two windows in one alert?
- What burn rate would exhaust a 30-day budget in two days?

```
# Burn rate: are we consuming budget faster than we can afford?
ALERT ErrorBudgetBurning
  IF (
    rate(errors[1h]) / rate(requests[1h]) > 14.4 * 0.001
    AND
    rate(errors[5m]) / rate(requests[5m]) > 14.4 * 0.001
  )
# 14.4x = burns 30-day budget in 2 days (page immediately)
# 6x    = budget gone in 5 days (ticket)
```

certs.eknathalabs.com

Multi-window multi-burn-rate alerts: fast (14.4x, 1h+5m windows) for acute incidents, slow (6x, 6h+30m windows) for gradual degradation. Reduces false positives from brief spikes while catching sustained issues. Use google/slo-generator or SLOTH for Kubernetes-native SLO alerting implementation.

📚 **Official docs**

- Google SRE Book: Service Level Objectives
- Google SRE Workbook: Alerting on SLOs
- Prometheus: Alerting rules

Q16 How do you design Grafana dashboards that are actually useful during incidents?

Dashboards

🎙 **Spoken answer · ~85 s**

During an incident, people need answers fast, so a good dashboard is organised around questions and follows a clear hierarchy. At the top, show service health at a glance: the golden signals, traffic, errors, latency percentiles, and saturation, plus SLO status and error budget, with red and green indicators using sensible thresholds. Below that, break down by dimension, such as per endpoint, region, version, or pod, so you can localise the problem. Then dependencies: database, cache, queue, and downstream service health, plus infrastructure resources. Mark deployments and config changes as annotations on graphs, because most incidents follow a change. Design principles: keep it uncluttered, with a small number of panels that matter; use consistent layouts and units across services so people learn them once; show rates and percentiles, not just averages; use the same time ranges and synchronised crosshairs; include links to logs, traces, and runbooks; use template variables for cluster, namespace, and service; and put the most important panels at the top left. Avoid vanity metrics and dozens of tiny graphs. Version dashboards as code with Grafonnet or Terraform for review and consistency. Test them in game days and incident reviews, and prune panels nobody uses. A great dashboard shortens time to diagnose.

Likely follow-ups

- Why add deployment annotations to graphs?
- How do you manage dashboards as code?

```
Row 1: Golden signals (error rate, P99 latency, traffic, saturation)
```

```
Row 2: Dependencies (downstream services, DB connection pool)
```

```
Row 3: Infrastructure (CPU, memory, pod restarts)
```

```
Row 4: Business (order rate, checkout success)
```

```
# Template variables for reuse:
```

certs.eknathalabs.com

```
sum(rate(requests_total{cluster="$cluster",service="$service"}[5m]))
```

```
# Add deployment annotations to see changes on timeline
```

Design for the incident responder woken at 3am: top row gives YES/NO health in 3 seconds, drill down by scrolling. Include links to runbooks. Use stat panels for current value, time-series for trends. Dashboard as code (Grafana provisioning ConfigMap or Terraform) keeps them version-controlled and auto-restores deleted dashboards.

📚 **Official docs**

- Grafana: Dashboard best practices
- Grafana documentation

Q17 What is the kube-prometheus-stack — what does it deploy?

Tools

🎙 **Spoken answer · ~78 s**

kube-prometheus-stack is a Helm chart, from the prometheus-community repository, that deploys a complete Kubernetes monitoring setup in one install. It includes the Prometheus Operator, which introduces custom resources such as Prometheus, Alertmanager, ServiceMonitor, PodMonitor, and PrometheusRule, so monitoring is configured declaratively; a Prometheus instance; Alertmanager; Grafana, with preloaded dashboards for cluster, node, workload, and Kubernetes components; node-exporter as a DaemonSet for host metrics; kube-state-metrics, which exposes the state of Kubernetes objects like deployments, pods, and nodes; and a large set of default alerting and recording rules covering common failure conditions, plus scrape configuration for the API server, kubelet, etcd, scheduler, and controller manager. Using the stack gives sensible defaults quickly. In production, I customise: set persistent storage and retention; size resources; configure Alertmanager receivers and routing; enable high availability with replicas; add long-term storage through Thanos or remote write; secure Grafana with SSO; disable default rules that don't apply and tune noisy ones; set selectors so teams can add their own ServiceMonitors; and manage the values in Git with GitOps. Remember that some managed Kubernetes platforms don't expose control plane components, so those scrape targets should be disabled.

Likely follow-ups

- What does kube-state-metrics provide that node-exporter does not?
- What does a ServiceMonitor do?

certs.eknathalabs.com

```
helm install kube-prom prometheus-community/kube-prometheus-stack -n monitoring -f values.yaml
```

```
# Deployed components:
```

```
Prometheus Operator  — manages Prometheus via CRDs (ServiceMonitor, PodMonitor, PrometheusRule)
```

```
Prometheus           — metrics collection
```

```
Alertmanager         — routing
```

```
Grafana              — dashboards (25+ pre-built)
```

```
kube-state-metrics   — K8s object metrics
```

```
node-exporter        — host metrics (DaemonSet)
```

```
prometheus-adapter   — custom metrics for HPA
```

Each team deploys their own ServiceMonitor alongside their Deployment — Prometheus auto-discovers it without central config changes. Pre-built dashboards cover nodes, K8s objects, etcd, coredns, and API server. Size Prometheus by series count: 1M series ≈ 3-4GB RAM. Use recording rules to pre-aggregate expensive queries at scale.

> 📚 **Official docs**
>
> - kubernetes-mixin (monitoring mixins)
> - Prometheus: Overview

certs.eknathalabs.com

Q18 What is Grafana Loki — how does it differ from Elasticsearch?

Tools

🎙 Spoken answer · ~85 s

Grafana Loki is a log aggregation system designed like Prometheus but for logs. The key difference from Elasticsearch is indexing. Elasticsearch builds a full-text inverted index over the content of every log line, making arbitrary searches fast but demanding significant CPU, memory, and disk, with a large index that often rivals the data itself. Loki indexes only metadata labels, such as app, namespace, and pod, and stores the log content as compressed chunks in object storage like S3 or GCS. Queries select streams using label matchers and then scan and filter the content at query time with LogQL, which supports line filters, parsing, and metric queries derived from logs. Consequences: Loki is much cheaper to run, simpler to operate, scales horizontally with cheap storage, and integrates natively with Grafana, so you can jump between metrics, logs, and traces using the same labels. The trade-offs: queries not narrowed by labels can be slower on large volumes, rich full-text analytics and aggregations are weaker, and high-cardinality labels are harmful, so you must design labels carefully and keep them few. Agents like Promtail, Grafana Alloy, or the OpenTelemetry Collector ship logs. I'd choose Loki for Kubernetes operations and debugging; Elasticsearch for heavy search or security analytics.

Likely follow-ups

- Why must Loki labels be low cardinality?
- What does LogQL let you do beyond searching?

```
helm install loki grafana/loki-stack -n monitoring
```

```
# LogQL examples:
```

```
{namespace="production",app="payments"} |= "error"
```

```
{app="api"} | json | status_code >= 500
```

```
rate({app="payments"} |= "payment_failed" [5m])  # metric from logs
```

Loki indexes only labels, not content — 10-20x cheaper than Elasticsearch. Tradeoff: full-text search is slower (scans lines grep-style). For most operational use cases (errors in a service/namespace/pod), Loki is sufficient. Elasticsearch excels at complex cross-service full-text search and compliance requirements. Managed: Grafana Cloud for zero-ops.

📚 Official docs

- Grafana documentation
- Grafana Loki documentation
- Elasticsearch reference

certs.eknathalabs.com

Q19 How do you monitor Kubernetes cluster health — what metrics and alerts matter most?

K8s obs

🎙 Spoken answer · ~77 s

Monitoring a Kubernetes cluster means covering the control plane, nodes, workloads, and the platform's add-ons. Control plane: API server request latency and error rate, etcd health, leader election and database size, scheduler and controller manager queue depth. Nodes: Ready status, CPU, memory, disk and inode pressure, network errors, and kubelet health. Workloads: pod restarts and CrashLoopBackOff, pods stuck Pending or in ImagePullBackOff, OOMKilled containers, deployment replicas available versus desired, container CPU throttling, memory working set against limits, and HPA status including when at maximum replicas. Capacity: resource requests versus allocatable, to know when to scale the cluster; persistent volume usage. Networking: DNS errors and latency in CoreDNS, ingress error rates, and certificate expiry. Alerts that matter: node not ready, pods crash looping, deployments failing to reach desired replicas, persistent volume nearly full, API server errors, etcd issues, certificate expiring soon, and cluster capacity exhaustion. Use kube-state-metrics, node-exporter, cAdvisor, and the kubelet endpoints through kube-prometheus-stack. Prioritise alerting on symptoms and workload health, and keep node-level alerts to those requiring action. Add events and audit logs, and review dashboards per team. Use SLOs for platform services themselves.

Likely follow-ups

- Which alerts would page someone at night?
- What does kube-state-metrics expose?

```
kube_node_status_condition{condition="Ready",status="true"} == 0  # node NotReady
```

```
increase(kube_pod_container_status_restarts_total[15m]) > 3       # crash loop
```

```
apiserver_request_duration_seconds{quantile="0.99"} > 1           # API server slow
```

```
etcd_server_has_leader == 0                                        # etcd no leader
```

```
kube_horizontalpodautoscaler_status_current_replicas ==
kube_horizontalpodautoscaler_spec_max_replicas  # HPA at max
```

```
absent(up{job="myapp"})  # target disappeared
```

HPA at maximum replicas is critical — autoscaling is exhausted and the service cannot handle more load. Certificate expiry (alert 30 days before) prevents entirely avoidable outages. absent() catches targets that stop reporting — a crash that takes the metrics endpoint with it would otherwise cause threshold alerts to miss the incident.

📚 Start here (official docs)

- Prometheus documentation

certs.eknathalabs.com

Q20 How do you detect CPU throttling in Kubernetes — why is it invisible?

K8s obs

🎙 Spoken answer · ~86 s

CPU throttling happens when a container with a CPU limit tries to use more CPU time than its limit allows within a scheduling period. Linux's CFS bandwidth control enforces limits in 100 ms periods: a limit of 500 millicores gives 50 ms of CPU time per period, and when a container exhausts that quota, all its threads are paused until the next period, even if the node has idle CPU. It's invisible because average CPU usage can look low, well below the limit, while short bursts, particularly in multi-threaded applications, hit the quota and cause latency spikes and slow responses. Dashboards that show average usage hide this. To detect it, use the cAdvisor metrics: container_cpu_cfs_throttled_periods_total divided by container_cpu_cfs_periods_total gives the percentage of periods in which the container was throttled, and container_cpu_cfs_throttled_seconds_total shows time lost. Alert when the throttled percentage is high, such as above twenty-five percent, and correlate with latency. Fixes: raise the CPU limit, or remove limits and rely on requests, which many teams do for latency-sensitive workloads, since requests already give proportional CPU shares; tune the thread pools and runtime settings to the container's CPU, for example GOMAXPROCS in Go or JVM container awareness; and right-size using observed usage, with tools like VPA recommendations.

Likely follow-ups

- Why does throttling occur when the node has idle CPU?
- Would you set CPU limits at all?

```
# Throttling ratio (alert if > 25%):
```

```
rate(container_cpu_cfs_throttled_seconds_total[5m]) /
rate(container_cpu_cfs_periods_total[5m])
```

```
# Why invisible: container does not error or crash
```

```
# It simply runs slower — requests take longer, no error logs
```

```
kubectl top pod myapp  # shows 0.8/1.0 CPU — looks fine
```

```
# But throttling metric shows 40% of periods throttled
```

The CFS scheduler allows a burst within the 100ms period, then throttles for the remainder. A pod using 0.8 CPU average on a 1.0 limit can still be 40% throttled if it spikes in the first 60ms of each period. Fix: increase CPU limit, reduce spike patterns, or remove CPU limits entirely (use requests only for scheduling). Always monitor throttling explicitly — CPU utilisation alone misses it.

📚 Official docs

- Kubernetes: Resource management for Pods & containers
- Linux kernel docs: CFS scheduler

certs.eknathalabs.com

Q21 How does GCP Managed Service for Prometheus differ from self-managed?

Cloud obs

🎙 **Spoken answer · ~87 s**

Google Cloud Managed Service for Prometheus, GMP, is Google's Prometheus-compatible managed backend, built on the same infrastructure as Monarch, Google's internal time-series database. Compared with self-managed Prometheus: with self-managed, you run the servers, size storage, handle high availability, upgrade, and add Thanos or similar for long-term retention and global views, which creates operational burden. With GMP, data is stored in Google's global datastore, which scales without you managing capacity, retains data for 24 months by default, and queries across clusters and projects through a single PromQL endpoint, with no sharding to think about. Collection can be managed, where Google runs the collectors as a lightweight DaemonSet configured through PodMonitoring resources, or self-deployed, where you keep your own Prometheus and use the Google-provided binary to write to GMP. It integrates with Cloud Monitoring alerting, and Grafana connects through the Prometheus data source with a proxy for authentication. Trade-offs: you pay per sample ingested, so cardinality management matters for cost; some features differ, such as rule evaluation arrangements; and you gain vendor dependency, though PromQL and exporters stay standard, which limits lock-in. I'd pick GMP on GKE when I want low operational overhead and global querying; self-managed when I need fine control, other clouds, or minimum cost at small scale.

Likely follow-ups

- How is GMP billed and how do you control cost?
- What is the difference between managed and self-deployed collection?

```
# Enable on GKE cluster:
```

```
gcloud container clusters update CLUSTER --enable-managed-prometheus
```

```
# Same ServiceMonitor CRDs, same PromQL, same Grafana integration
```

```
# Backend: Cloud Monitoring (15 month retention, no infra to manage)
```

```
gcloud monitoring metrics list --
filter="metric.type=starts_with(prometheus.googleapis.com)"
```

GMP is Prometheus-compatible (same SDKs, PromQL, ServiceMonitors) but stores in Cloud Monitoring backend — no Prometheus StatefulSets, no storage management, no HA configuration. For GKE-native teams, GMP reduces operational overhead significantly. Self-managed Prometheus: full control, open standards, multi-cloud/on-prem, no per-series ingestion cost. For a GCP-native platform team, GMP is typically the right default.

📚 **Official docs**

- Google Cloud Managed Service for Prometheus
- Prometheus: Overview

certs.eknathalabs.com

Q22 How do you use Cloud Logging and Cloud Trace for full-stack observability?

Cloud obs

🎙 Spoken answer · ~84 s

Cloud Logging and Cloud Trace combine to give full-stack visibility on Google Cloud. Cloud Logging collects logs automatically from GKE, Cloud Run, Compute Engine, and Google services, with structured JSON logs parsed into fields; you query with the Logging query language, build log-based metrics and alerts, and route logs through sinks to BigQuery, Cloud Storage, or Pub/Sub, with exclusion filters to control cost. Cloud Trace collects latency data from instrumented applications, typically through OpenTelemetry exporters, and shows traces with spans, latency distributions, and analysis reports. The integration: when applications log the trace and span IDs in the fields Cloud Logging recognises, such as logging.googleapis.com/trace and spanId, the console links log entries to the trace, so from a slow trace you can open the exact logs for the request, and from an error log you can jump into its trace. Add Cloud Monitoring for metrics, dashboards, SLOs, and alerting, with Error Reporting to group exceptions, and Cloud Profiler for continuous profiling. A typical workflow: an SLO alert fires, a dashboard narrows it to a service, Trace identifies the slow downstream span, and the correlated logs reveal the root cause. Ensure consistent context propagation across services, set sampling sensibly, and control log volume.

Likely follow-ups

- How do logs link to traces in Cloud Logging?
- What is a log-based metric?

```
# Link logs to traces — set trace field in log entries:
```

```
{"trace": "projects/PROJECT/traces/TRACE_ID", "spanId": "SPAN_ID"}
```

```
# Cloud Logging auto-links to Cloud Trace when trace field is present
```

```
# Log-based metric from error pattern:
```

```
gcloud logging metrics create payment_errors \
```

```
  --log-filter='jsonPayload.event="payment_failed"'
```

```
# Error Reporting groups exceptions automatically
```

```
gcloud beta error-reporting events list --project=PROJECT
```

GCP observability stack: Cloud Logging ← container stdout (auto), Cloud Trace ← OTLP exporter, Cloud Profiler ← sampling profiler (zero overhead). Log entries with trace field show a "View trace" link in Cloud Logging console — one click jumps to the full distributed trace for that request. Log-based metrics extract signal from logs into Cloud Monitoring without application code changes.

certs.eknathalabs.com

📚 **Official docs**

- Cloud Logging
- Cloud Trace

Q23 A production alert fires — walk through your structured incident response.

Incident

🎙 **Spoken answer · ~85 s**

When a production alert fires, I follow a structured process. Acknowledge it so others know someone is on it, and quickly assess severity and user impact: is this a real, customer-facing issue? If significant, declare an incident, assign roles, such as incident commander, communications lead, and operations lead, and open a dedicated channel. Prioritise mitigation over root cause: the goal is to restore service. Check recent changes first, as deployments, config changes, and feature flags cause most incidents; if there's a suspect change, roll back or turn off the flag. Otherwise, use dashboards and the golden signals to localise: which service, region, or dependency? Use traces and logs to confirm. Apply the fastest safe mitigation: roll back, fail over, scale up, shed load, or disable a feature. Communicate regularly to stakeholders and a status page, with what is known, impact, and next update time. Keep a timeline of actions in the channel for the later review. Once service is stable, confirm recovery against the SLIs, downgrade the incident, and hand off if the shift ends. Afterwards, hold a blameless postmortem, identify root causes and contributing factors, and track action items to completion. Escalate early rather than late when stuck, and avoid making multiple simultaneous changes.

Likely follow-ups

- Why prioritise mitigation over root cause?
- What roles do you assign in a major incident?

```
Phase 1 — Detect (< 5 min): acknowledge alert, assess impact, open Slack channel
Phase 2 — Triage (5-15 min):
  Golden signals dashboard: which signal is anomalous?
  Recent changes: kubectl rollout history, recent deploys, config changes
  Form a hypothesis before taking action
Phase 3 — Mitigate:
  Rollback first if recent deploy (fastest recovery)
  Scale up if saturation issue
  Disable feature flag if new feature causing errors
```

certs.eknathalabs.com

```
Phase 4 — Resolve: confirm metrics returned to baseline, post status update
```

```
Phase 5 — Postmortem: schedule within 48 hours
```

One incident commander coordinates (does not necessarily fix). Other engineers investigate. Rule: make one change at a time — simultaneous changes make it impossible to know what fixed the issue. Always rollback first if unsure — a known-good state beats an unknown state.

📚 **Official docs**

- Google SRE Book: Managing Incidents
- Google SRE Workbook: Incident Response

Q24 How do you write a blameless postmortem — what structure matters?

Incident

🎙 **Spoken answer · ~82 s**

A blameless postmortem examines an incident to learn and improve, focusing on systems and processes instead of individuals. The premise is that people act reasonably given the information and incentives they had, so blame hides the real causes and discourages honest reporting. Structure that matters: a summary with impact, duration, and severity; the user and business impact, quantified, such as errors, revenue, and SLO budget burned; a detailed timeline with timestamps of detection, escalation, actions, and resolution; the root cause and contributing factors, using techniques like the five whys while avoiding a single cause, since complex failures usually involve several; what went well, such as fast detection or good communication; what went poorly; where we got lucky; metrics such as time to detect, time to mitigate, and time to recover; and action items that are specific, with owners, priorities, and due dates, covering prevention, detection, and mitigation. Hold the review soon after, while memory is fresh, with all involved, facilitated neutrally. Write in neutral language, using roles rather than blaming names. Publish it widely so other teams learn, and track action items to completion, otherwise the exercise is wasted. Review trends across incidents to find systemic weaknesses.

Likely follow-ups

- Why avoid a single root cause?
- How do you ensure action items get completed?

```
1. Incident summary (severity, duration, user impact)
```

```
2. Timeline (who did what, when)
```

```
3. Root cause (5 Whys — get to systemic cause)
```

```
4. Contributing factors
```

```
5. What went well
```

certs.eknathalabs.com

```
6. Action items (specific owner + due date)
```

```
# 5 Whys example:
```

```
Service crashed → OOM at 2x load → Marketing campaign not communicated
```

```
→ No cross-team process → Fix: establish the process (not just raise memory limits)
```

Blameless: goal is systemic improvement, not fault assignment. People make mistakes — well-designed systems prevent mistakes from causing incidents. Every postmortem must produce action items with owners and due dates. Track completion rate — below 80% means the process is broken and incidents will recur. This is a Principal-level responsibility.

📚 **Official docs**

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture
- Atlassian: How to run a blameless postmortem

Q25 What is an SLO dashboard — how do you visualise error budget in Grafana?

Tools

🎙 **Spoken answer · ~88 s**

An SLO dashboard shows how the service is performing against its objectives and how much error budget remains. It typically contains: the current SLI value over the SLO window, for example thirty-day availability, next to the target; the remaining error budget as a percentage, shown as a gauge or stat panel with colour thresholds, green when plenty remains, yellow as it shrinks, and red when nearly gone; a time series of budget consumption over the window, so you can see when it was spent and the trend; the burn rate over multiple windows, such as one hour and six hours, showing how fast the budget is being used right now; and annotations marking deployments and incidents. In Grafana, build it on Prometheus recording rules that precompute the good-event and total-event ratios over different windows, because querying thirty days of raw data in every panel is slow and expensive. Remaining budget is one minus the observed error ratio divided by the allowed error ratio. Use template variables for service and environment so one dashboard serves all services, and generate them from SLO definitions with tools like Sloth or Pyrra, which also create the alert rules. Put it on the team's main page and review it in planning, so the budget actually drives decisions.

Likely follow-ups

- Why use recording rules for SLO queries?
- How do tools like Sloth help?

certs.eknathalabs.com

```
# Error budget remaining panel (PromQL):
```

```
1 - (sum(increase(requests_total{code=~"5.."}[30d])) / (sum(increase(requests_total[30d]))
* 0.001))
```

```
# Error budget burn rate panel:
```

```
rate(errors[1h]) / rate(requests[1h]) / 0.001  # ratio to allowed rate
```

```
# Colour thresholds: green > 50%, yellow 10-50%, red < 10%
```

An SLO dashboard has three panels: current SLI value vs target (are we in SLO right now?), error budget remaining % (how much budget is left this month?), and burn rate over time (are we accelerating or decelerating?). Green error budget with stable burn rate = healthy, deploy freely. Red budget with high burn rate = freeze deployments, fix reliability. The dashboard makes the deployment decision data-driven.

📚 **Official docs**

- Google SRE Book: Service Level Objectives
- Google SRE Workbook: Implementing SLOs
- Google SRE Book: Embracing Risk (error budgets)

certs.eknathalabs.com

Q26 How do you export custom application metrics from a Kubernetes pod to Prometheus?

K8s obs

🎙 Spoken answer · ~86 s

To expose custom application metrics, instrument the application with a Prometheus client library for its language, such as prometheus_client for Python, client_golang for Go, or Micrometer for Java. Define metrics, counters for events, gauges for current values, and histograms for durations, with clear names, base units, and a small, bounded set of labels, then expose them on an HTTP /metrics endpoint, either by the library's built-in server or middleware. In Kubernetes, the pod serves the endpoint on a named container port, and a Service exposes it. Then tell Prometheus to scrape it. With the Prometheus Operator, create a ServiceMonitor that selects the Service by labels and names the port and path, or a PodMonitor to select pods directly; make sure the monitor carries the labels that Prometheus's selector expects. Without the Operator, use pod annotations such as prometheus.io/scrape with Prometheus's kubernetes_sd configuration. Verify in the Prometheus targets page that the target is up, and query the metric. Follow naming conventions, like a _total suffix for counters and _seconds for durations, and avoid high-cardinality labels such as user IDs. For short-lived jobs, use the Pushgateway or write to a textfile collector. Alternatively, use OpenTelemetry metrics through a collector. Document metrics and add them to dashboards and alerts.

Likely follow-ups

- What is the difference between a ServiceMonitor and a PodMonitor?
- Why avoid user IDs as labels?

```
# Python instrumentation:
```

```
from prometheus_client import Counter, Histogram, start_http_server
```

```
requests_total = Counter("myapp_requests_total","Total requests",["status"])
```

```
latency = Histogram("myapp_request_duration_seconds","Request latency")
```

```
# Expose metrics port in Service:
```

```
ports: [{name: metrics, containerPort: 8081}]
```

```
# ServiceMonitor CRD (Prometheus Operator picks this up):
```

```
spec:
```

```
  selector: {matchLabels: {app: myapp}}
```

```
  endpoints: [{port: metrics, interval: 30s}]
```

```
# Verify: kubectl port-forward svc/prometheus 9090 → check /targets
```

certs.eknathalabs.com

ServiceMonitor is the GitOps-friendly pattern: each team deploys their own ServiceMonitor alongside their Deployment. Prometheus auto-discovers it without central config changes. Metric naming conventions: myapp_http_requests_total (counter), myapp_request_duration_seconds (histogram), myapp_active_connections (gauge). Unit in name, snake_case, _total suffix for counters.

📚 **Official docs**

- Prometheus: Overview

Q27 How do you set up PagerDuty routing from Alertmanager?

Alerting

🎙 **Spoken answer · ~85 s**

Integrating PagerDuty with Alertmanager starts in PagerDuty: create a service, choose the Prometheus or Events API v2 integration, and copy the integration, or routing, key. In Alertmanager's configuration, define a receiver with a pagerduty_configs block containing that key, stored as a secret rather than in plain text, and optionally a description and details built from templates that include the alert name, severity, summary, dashboard link, and runbook URL. Then set up the route tree to decide which alerts go where: route by labels such as severity and team, so critical alerts from the payments team go to the payments PagerDuty receiver, while warnings go to Slack or a ticket queue. Use group_by to batch related alerts, with group_wait and group_interval tuned for timeliness, and repeat_interval to avoid over-paging. Add inhibition rules so a cluster-level alert suppresses its symptoms. Alertmanager sends resolved notifications automatically, which PagerDuty uses to auto-resolve incidents. In PagerDuty, configure escalation policies and on-call schedules so each service pages the right people, with escalation if unacknowledged. Test end to end with a test alert using amtool or a synthetic rule, and also keep a deadman's switch to detect broken alerting. Keep the configuration in version control, and review routing regularly as teams change.

Likely follow-ups

- How do you test the whole alert path?
- How are alerts auto-resolved in PagerDuty?

```
receivers:
```

```
- name: pagerduty
```

```
  pagerduty_configs:
```

```
  - routing_key: <INTEGRATION_KEY>   # from PagerDuty service integration
```

```
    description: "{{ .CommonAnnotations.description }}"
```

```
    severity: "{{ .CommonLabels.severity }}"
```

certs.eknathalabs.com

```
    details:
```

```
      runbook: "{{ .CommonAnnotations.runbook_url }}"
```

```
      dashboard: "{{ .CommonAnnotations.dashboard_url }}"
```

Store the integration key in a Kubernetes Secret referenced by Alertmanager. Route: critical severity → PagerDuty (wakes people up), warning → Slack (async review). Runbook and dashboard URLs in every alert — they appear in the PagerDuty incident, giving on-call engineers immediate context. This alone reduces MTTA by 50%.

📚 **Official docs**

- Prometheus: Alertmanager

Q28 What are Grafana template variables — how do you use them for multi-cluster dashboards?

Dashboards

🎙 **Spoken answer · ~83 s**

Grafana template variables make dashboards dynamic: instead of hardcoding values like a cluster name, you define variables that appear as dropdowns and are substituted into panel queries. A variable can be a query, populated by a data source query such as label_values(kube_pod_info, cluster); a custom list; a data source variable, to switch between Prometheus instances; an interval; or a constant. Panels reference them with syntax like $cluster, so one dashboard serves every cluster, namespace, or service. For multi-cluster dashboards: add a cluster variable, either selecting a data source per cluster or, better, using a label such as cluster attached to all metrics via external_labels or a central store like Thanos, Mimir, or GMP. Then chain variables so that they depend on each other: namespace filtered by the selected cluster, then pod filtered by namespace, by using the earlier variable inside the later query. Enable multi-value and the include-all option for comparisons, and use regex matchers in queries, such as cluster=~"$cluster". Use repeat panels or rows to produce a panel per selected value. Practices: keep queries performant, give variables clear names, set sensible defaults, and avoid enormous value lists. Manage dashboards as code so that variables stay consistent across teams.

Likely follow-ups

- How do chained variables work?
- How do you make one dashboard cover many clusters?

```
# Variable: cluster (auto-populated from metric labels)
```

```
Name: cluster
```

```
Type: Query
```

certs.eknathalabs.com

```
Query: label_values(kube_node_info, cluster)
```

```
# Use in panel:
```

```
sum by (namespace) (rate(requests_total{cluster="$cluster"}[5m]))
```

```
# $__rate_interval: auto-adjusts to time range and scrape interval
```

```
rate(requests_total[$__rate_interval])  # better than hardcoded [5m]
```

Template variables make one dashboard serve all clusters. Chained variables: selecting a cluster narrows the namespace dropdown. Data links add drill-down: click a pod in the overview to jump to the pod-detail dashboard with that pod pre-selected. Use $__rate_interval instead of hardcoded time windows — it adjusts automatically to the dashboard time range and scrape interval.

📚 **Official docs**

- Grafana documentation

Q29 Distributed traces show high latency but you cannot find the cause — how do you debug?

Tracing

🎙 **Spoken answer · ~89 s**

When traces show high latency but no obvious culprit, I look for time that isn't attributed to any span. First, examine the trace's structure: gaps between spans often mean time spent in uninstrumented code, queueing, thread pool or connection pool waits, garbage collection pauses, or network hops. Check whether the parent span is much longer than the sum of its children. Add instrumentation to the suspicious area, such as database client calls, cache access, serialisation, and outbound HTTP, and use manual spans where automatic instrumentation doesn't reach. Compare slow and fast traces for the same endpoint to find what differs: a specific tenant, region, version, payload size, or cache miss. Use the attributes on spans to filter and group. Correlate with other signals: metrics for CPU throttling, memory pressure, saturation, and connection pool exhaustion; logs for the slow trace IDs; and continuous profiling to see where the CPU time goes. Consider infrastructure causes, such as DNS lookups, TLS handshakes, noisy neighbours, retries, and timeouts hidden by clients, and clock skew between hosts that distorts span timing. Also check sampling: you might be seeing unrepresentative traces, so use tail-based sampling to capture the slow ones. Once the cause is confirmed, fix it and add an alert or span so it is visible next time.

Likely follow-ups

- What do gaps between spans usually indicate?
- How does continuous profiling complement tracing?

```
# Step 1: Find slow trace — filter by duration in Jaeger/Tempo
```

certs.eknathalabs.com

```
# Step 2: Inspect trace waterfall — wide span = slow, gap = wait
```

```
db.statement, http.url, db.row_count, error.message  # span attributes
```

```
# Step 3: Cross-reference logs with trace_id
```

```
{service="payments"} | json | trace_id="abc123"
```

```
# Step 4: Check resource metrics at time of slow trace
```

```
container_cpu_cfs_throttled_seconds_total  # was the pod throttled?
```

Trace waterfall: width = duration, indentation = parent-child. Gap between parent starting and first child = time in parent before the downstream call (serialisation, connection pool wait). Wide span with no children = bottleneck is in that service. Span attributes (db.statement, http.url) give the exact SQL or API call. Cross-correlate with logs using trace_id to see application-level context at that exact moment.

📚 **Official docs**

- OpenTelemetry: Signals (traces, metrics, logs)

Q30 What is a runbook — how do you write one that actually helps at 3am? Incident

🎙 **Spoken answer · ~88 s**

A runbook is a documented procedure for responding to a specific alert or operational task. A good one works for someone tired, stressed, and possibly unfamiliar with the system at 3am, so it must be clear, short, and actionable. Structure: a title matching the alert name, linked directly from the alert; a one-line description of what the alert means and the user impact; severity and who to escalate to; the quick checks first, with the exact dashboard links and copy-pasteable commands or queries; a decision flow, if this then that, rather than long prose; the mitigation steps in order, such as rollback, restart, scale, or failover, with the expected result of each and how to verify success; a clear stop point stating when to escalate and to whom; and known causes from past incidents. Write it with concrete commands, not vague advice, and include safety warnings for risky actions. Keep it up to date: assign an owner, review it after every incident and during game days, test it by having someone new follow it, and delete obsolete ones. Store it somewhere accessible even when your systems are down. Automate steps where possible, converting stable runbook steps into scripts or auto-remediation. The best runbooks gradually disappear because the problem is automated or fixed.

Likely follow-ups

- What should every runbook link from the alert contain?
- When should a runbook step be automated?

```
# Runbook structure:
```

certs.eknathalabs.com

```
## Alert: PaymentServiceHighErrorRate

### What this means: error rate above 0.1% SLO threshold

### Severity: P1 if > 1%, P2 if 0.1-1%

### Triage steps:

1. Open dashboard: [link]

2. kubectl get pods -n payments | grep -v Running

3. kubectl logs -l app=payments --tail=50 | grep ERROR

4. Check recent deployments: kubectl rollout history deployment/payments

### Mitigation: kubectl rollout undo deployment/payments

### Escalation: payments-oncall → payments-lead → CTO
```

A good runbook assumes the reader has been woken up at 3am and has never seen this alert before. Numbered steps, exact commands to copy-paste, expected outputs, decision branches ("if step 3 shows no errors, skip to step 7"). Link to dashboards with time pre-set. Include escalation path and communication templates. Update the runbook after every incident — add the debugging steps that actually worked.

> 📚 **Start here (official docs)**
>
> - Prometheus documentation

certs.eknathalabs.com

Q31 What are the Prometheus metric types: counter, gauge, histogram and summary?

Metrics

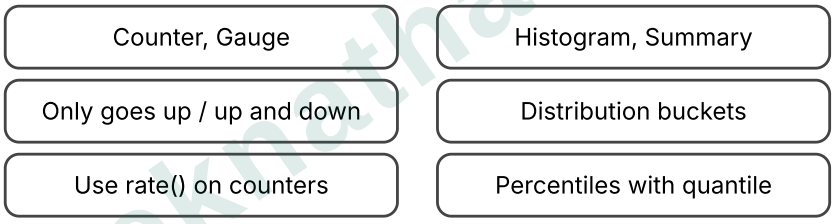
🎙 **Spoken answer · ~60 s**

A counter is a cumulative value that only increases, or resets to zero on restart, such as requests served; its raw value is rarely useful, so I apply rate or increase, which handle resets. A gauge is a value that can go up and down, such as memory in use, queue length or temperature, and is read directly or with functions like avg_over_time. A histogram counts observations into configurable buckets and also keeps a sum and count; it allows calculating quantiles at query time with histogram_quantile and, importantly, aggregating across instances, so I prefer it for latency. A summary calculates quantiles on the client side, which is accurate for a single instance but cannot be aggregated across many. Bucket choice matters: they should surround the SLO thresholds. Newer native histograms improve resolution and cost. Choosing the right type determines whether the numbers are valid.

Likely follow-ups

- Why can histograms be aggregated but summaries not?
- How do you pick bucket boundaries?

🗺 **Metric types**



Counters only increase, gauges go up and down, histograms bucket observations for aggregable quantiles, summaries compute quantiles client-side.

```
rate(http_requests_total[5m])
```

```
histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
```

Interview tip: Always use rate() on counters.

📚 **Official docs**

- Prometheus: Histograms and summaries
- Prometheus: Overview

certs.eknathalabs.com

Q32 What is cardinality in metrics and how do you control it?

Metrics

🎙 Spoken answer · ~61 s

Each unique combination of metric name and label values creates a separate time series that the server must store and index. Labels with unbounded values, such as user IDs, email addresses, request URLs with identifiers or container IDs, can create millions of series, causing high memory use, slow queries and big bills in hosted systems. I control cardinality by designing labels from small, bounded sets such as method, route template and status class, never raw paths; dropping or aggregating unneeded labels with relabelling at scrape time; setting sample and series limits per target so one bad service cannot take down the monitoring; and using logs or traces for high-cardinality detail. I find offenders with queries that count series by metric and by label, and track series count as a metric with an alert. Churn from short-lived pods also contributes, so I watch both active and created series.

Likely follow-ups

- Which labels are dangerous?
- How do you find high-cardinality metrics?

Cardinality is the number of unique label combinations; high-cardinality labels like user IDs explode memory and cost.

```
topk(10, count by (__name__)({__name__=~".+"}))
```

```
metric_relabel_configs: - action: labeldrop regex: pod_uid
```

Interview tip: Never label by unbounded values.

📚 Official docs

- Prometheus: Metric and label naming

certs.eknathalabs.com

Q33 How do recording rules and federation help scale Prometheus?

Metrics

🎙 Spoken answer · ~59 s

Recording rules evaluate an expression at a regular interval and store the result as a new time series. Dashboards and alerts that use heavy expressions, such as quantiles over many series, then read the cheap precomputed series, making them fast and consistent. A naming convention like level:metric:operations keeps them understandable. For scale, a single Prometheus has limits on series count and retention, so I shard by team or environment and aggregate globally. Federation lets a central Prometheus scrape selected aggregated series from others, which works for summary views but is not meant to copy everything. For long-term storage and a global view, remote write to systems such as Thanos, Cortex, Mimir or a managed service is the common approach, with downsampling for older data. I keep high-resolution data local for alerting, since it must stay available when the network is not.

Likely follow-ups

- What is the naming convention for recorded series?
- When use remote write instead of federation?

Recording rules precompute expensive queries; federation or remote write aggregates data across servers.

```
groups:
- name: slo
  rules:
  - record: job:http_requests:rate5m
    expr: sum by (job) (rate(http_requests_total[5m]))
```

Interview tip: Dashboards should query recorded series.

📚 Official docs

- Prometheus: Recording rules
- Prometheus: Federation
- Prometheus: Overview

certs.eknathalabs.com

Q34 What is the USE method and how does it differ from RED?

Metrics

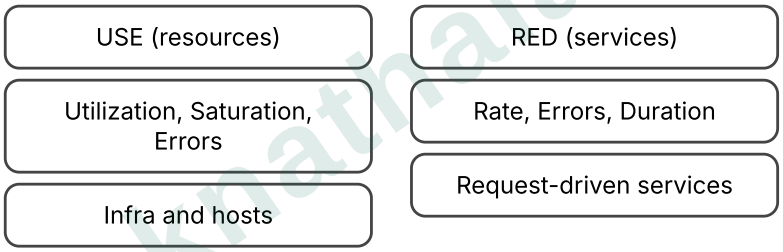
🎙 Spoken answer · ~56 s

The USE method, from Brendan Gregg, examines every resource, such as CPU, memory, disks and network interfaces, for utilisation, meaning how busy it is, saturation, meaning queued or waiting work beyond capacity, and errors. It is a checklist for finding resource bottlenecks quickly. The RED method, from Tom Wilkie, suits microservices and focuses on rate of requests, number of errors and duration or latency. It describes the experience of the service's users. They complement each other: RED tells me a service is slow or failing, and USE helps find which underlying resource is the cause. The four golden signals from the SRE book, latency, traffic, errors and saturation, combine both ideas. In practice I build service dashboards with RED, infrastructure dashboards with USE, and link them so an engineer can move from symptom to cause.

Likely follow-ups

- Which method tells you the cause?
- How do the golden signals relate?

🗺 USE vs RED



USE (utilisation, saturation, errors) analyses resources; RED (rate, errors, duration) analyses request-driven services.

```
node_cpu_seconds_total
```

```
rate(http_requests_total[5m])
```

Interview tip: USE for hosts, RED for services.

📚 Official docs

- Google SRE Book: Monitoring Distributed Systems

certs.eknathalabs.com

Q35 How do you measure and alert on saturation, and why is it hard?

Metrics

🎙 **Spoken answer · ~63 s**

Saturation means the system has more work than it can process immediately, so work queues up. It is hard because many resources saturate in non-obvious ways and utilisation alone misleads: a CPU at seventy percent average can still have throttled threads or a long run queue, and a thread pool can be exhausted while CPU is idle. Useful measures include run queue length and load relative to cores, CPU throttling in containers, memory pressure and swap activity, disk I/O queue depth and wait time, connection pool wait time, thread pool queue sizes, and message queue backlog. For capacity-like resources such as disk space, I use predictive alerts based on the trend, for example a linear forecast that the disk fills within four hours, which gives time to act. I treat sustained saturation as a leading indicator of latency and errors and often alert on it with lower urgency than user-facing symptoms.

Likely follow-ups

- Why is utilisation not enough?
- What is a predictive alert?

Saturation is how full a resource is; measure queue depth, throttling, run queue and utilisation against limits, and alert predictively.

```
node_load1 / count(node_cpu_seconds_total{mode="idle"}) without (cpu, mode)
```

```
predict_linear(node_filesystem_avail_bytes[6h], 4*3600) < 0
```

Interview tip: Queues reveal saturation before errors do.

📚 **Start here (official docs)**

- Prometheus documentation

certs.eknathalabs.com

Q36 How do you design log levels and what should be logged at each level?

Logging

🎙 Spoken answer · ~60 s

Levels communicate importance and control volume. Error means an operation failed in a way that someone may need to act on, and should include context and the exception. Warning marks unexpected but handled situations, such as a retry or fallback, that might indicate a developing problem. Info records significant lifecycle and business events, like service start, configuration loaded or an order placed, at a rate that is affordable in production. Debug and trace are detailed diagnostics, normally disabled in production, or enabled temporarily per component or request. I avoid logging inside tight loops, avoid logging the same error at every layer as it propagates, and never log secrets, tokens or personal data. Logs should be structured with consistent field names, a correlation or trace identifier and enough context to reproduce the issue. Reviewing logs periodically for noise keeps the signal useful and costs under control.

Likely follow-ups

- What should every error log contain?
- Why not log the same exception at each layer?

Error for failures needing attention, warn for recoverable anomalies, info for key business events, debug for diagnostics; avoid noise and secrets.

```
logger.info('order_created', order_id=oid, amount=amt)
```

```
logger.error('payment_failed', exc_info=True, order_id=oid)
```

Interview tip: Every error log should be actionable.

📚 Official docs

- OpenTelemetry: Logs

certs.eknathalabs.com

Q37 How do you handle sensitive data and compliance in logs?

Logging

🎙 **Spoken answer · ~62 s**

Logs are often copied to many systems and retained for long, so sensitive data in them becomes a liability under privacy and security rules. The first line of defence is not logging it: developers avoid printing request bodies, headers with tokens, passwords, card numbers and personal identifiers, with libraries or lint rules that help. Where fields are needed, I mask, truncate or hash them at the source, and add redaction rules in the collection pipeline as a safety net, using pattern matching for known formats. Access to logs is restricted by role, with audit trails of who searched what, and sensitive streams such as authentication or payment logs get stricter controls and shorter retention. Retention periods match legal requirements and operational need, and deletion requests must be feasible, which another reason for keeping personal data out. Encrypt in transit and at rest, and test the pipeline for leaks periodically.

Likely follow-ups

- Why redact at the source?
- How do retention rules help compliance?

Do not log secrets or personal data; mask or hash at source, restrict access, set retention and audit log access.

```
processors: - redaction: allowed_keys: [user_id, status] blocked_values: ['\\b\\d{16}\\b']
```

Interview tip: Redact at the source, not later.

📚 **Start here (official docs)**

- Prometheus documentation

certs.eknathalabs.com

Q38 What is log sampling and when is it appropriate?

Logging

🎙 Spoken answer · ~61 s

At high traffic, logging every successful request at info level can be enormously expensive and mostly redundant. Sampling reduces volume by keeping a fraction, perhaps five percent of routine events, while always retaining errors, warnings and slow requests. To keep logs useful, I sample consistently by request or trace identifier so that either all log lines of a request are kept or none, which preserves the story; random per-line sampling produces fragments. Rate-limited logging prevents a failing loop from emitting millions of identical lines, with a counter of suppressed messages. Metrics should cover what sampled logs omit, since counts and rates need to be exact; I never derive metrics from sampled logs. Dynamic sampling can raise the rate temporarily during incidents. Tail-based approaches in a collector decide after seeing the whole request, keeping interesting ones. Sampling should be documented so responders know what might be missing.

Likely follow-ups

- Why sample by request ID?
- Why not compute metrics from sampled logs?

Keep a representative fraction of high-volume low-value logs while always keeping errors and rare events.

```
if hash(request_id) % 100 < 5: logger.debug(...)
```

Interview tip: Sample by request ID to keep complete flows.

📚 Official docs

- OpenTelemetry: Sampling

certs.eknathalabs.com

Q39 How do you query logs effectively in Loki with LogQL or in Elasticsearch?

Logging

🎙 Spoken answer · ~62 s

In Loki, only labels are indexed, so a query starts with a stream selector using labels like app, namespace and environment to narrow to the relevant streams, then applies line filters such as contains or regex, then parsers like json or logfmt to extract fields, and finally filters or aggregates on extracted fields. The earlier I narrow, the less data is scanned, so I avoid selectors that match everything and keep the time range tight. LogQL can also compute metrics from logs, like error rates by route, useful for ad hoc analysis or alerts. In Elasticsearch, the whole text is indexed, so queries use the query DSL or KQL on fields, with time filters first; keyword fields are exact, while text fields are analysed. For both systems I make sure logs are structured with consistent fields, which makes queries simple, and create saved queries and dashboards for common investigations.

Likely follow-ups

- Why are labels special in Loki?
- When convert logs to metrics?

Filter by indexed labels first, then line filters and parsers; avoid broad regexes over long ranges.

```
{app="api", env="prod"} |= "timeout" | json | status >= 500
```

```
sum by (route) (rate({app="api"} |= "error" [5m]))
```

Interview tip: Labels select streams; filters scan text.

📚 Official docs

- Grafana Loki documentation
- Elasticsearch reference

certs.eknathalabs.com

Q40 What are the trade-offs between agent-based, sidecar and direct logging pipelines?

Logging

🎙 Spoken answer · ~65 s

A node-level agent, running as a daemonset, tails container log files on each node and ships them, which uses few resources and needs no changes to applications, making it the default. A sidecar per pod suits cases where an application writes to files, needs special parsing or has to send to a different destination, but adds resource overhead for every pod and operational complexity. Direct shipping from the application via a logging SDK avoids files but couples the app to the backend, adds latency and can lose logs or block the application if the backend is down. Whatever the approach, I design for backpressure and failure: use disk buffering in agents, bounded queues, retries with backoff, and drop policies that protect the application. A central aggregation layer, such as a collector or message queue, absorbs bursts, allows routing and filtering, and decouples producers from storage. I monitor the pipeline's own health, including dropped and delayed logs.

Likely follow-ups

- When is a sidecar justified?
- How do you protect apps from logging backpressure?

Node agents are efficient and simple; sidecars isolate per-app needs; direct SDK shipping couples the app to the backend and risks loss.

```
kubectl get ds -n logging
```

```
fluent-bit -c /etc/fluent-bit.conf
```

Interview tip: Buffer on disk to survive outages.

📚 Official docs

- Kubernetes: Sidecar containers

certs.eknathalabs.com

Q41 What is trace sampling and how do head-based and tail-based sampling differ?

Tracing

🎙 **Spoken answer · ~59 s**

Storing every trace at high traffic is too expensive, so we sample. Head-based sampling makes a decision when the trace begins, such as keep ten percent, and propagates the decision to all services. It is simple and cheap but blind: it may discard the rare slow or failed request I most want. Tail-based sampling waits until the trace is complete, usually in a collector, and then applies policies, such as keep all traces with errors, keep those slower than a threshold, and a small random fraction of the rest. It captures the interesting traces but requires buffering all spans of a trace in memory and routing all spans for a trace to the same collector instance, which adds complexity and cost. Many systems combine them: a modest head sample plus tail rules. Metrics derived from spans before sampling keep rates accurate.

Likely follow-ups

- Why is head sampling blind?
- What does tail sampling require?

Head sampling decides at the start of a trace; tail sampling decides after seeing the whole trace, so it can keep errors and slow traces.

```
processors:
  tail_sampling:
    policies:
      - name: errors
        type: status_code
        status_code: {status_codes: [ERROR]}
```

Interview tip: Tail sampling needs all spans of a trace in one place.

📚 **Official docs**

- OpenTelemetry: Sampling

certs.eknathalabs.com

Q42 What is context propagation and what breaks it?

Tracing

🎙 **Spoken answer · ~65 s**

For a trace to span multiple services, each service must receive the trace context from the caller and pass it on to callees. The W3C Trace Context standard defines the traceparent header with the trace identifier, parent span identifier and flags, and baggage can carry extra key-value data. Instrumentation libraries inject and extract these automatically for common HTTP and gRPC clients and servers. Propagation breaks at boundaries the library does not cover: message queues, where context must be placed in message headers; background jobs and thread pools, where the context does not automatically move to another thread; proxies, gateways or load balancers that strip unknown headers; and services using different propagation formats, such as older B3 headers. Symptoms are broken traces, orphan spans and traces that begin in the middle. I fix these by configuring consistent propagators, instrumenting messaging and async hand-offs, and checking gateways, then verifying with a test request through the full path.

Likely follow-ups

- Which header carries trace context?
- Why do queues break traces?

Trace identifiers travel in headers or message metadata so spans link across services; breaks occur at async boundaries, proxies and unsupported libraries.

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
```

```
propagators: [tracecontext, baggage]
```

Interview tip: Instrument queues and thread pools explicitly.

📚 **Start here (official docs)**

- Prometheus documentation

certs.eknathalabs.com

Q43 How do you use span attributes, events and links wisely?

Tracing

🎙 Spoken answer · ~60 s

Attributes are key-value metadata that make spans searchable and filterable, such as route template, status code, database system, tenant or feature flag. I follow OpenTelemetry semantic conventions for names so tools understand them, use bounded values for anything I plan to group by, and avoid putting secrets or huge payloads in them. Events are timestamped annotations within a span, for example a retry, cache miss or exception, that explain what happened during its lifetime without creating a full child span. Links connect spans that are related but not parent and child, such as a batch consumer processing many messages, each from a different producer trace, or a retry that references the original attempt. Good instrumentation creates spans for meaningful operations, like outgoing calls and significant internal steps, but not for every function, which adds overhead and noise. Consistent attributes make traces answer questions quickly.

Likely follow-ups

- What are semantic conventions?
- When use a link instead of a parent?

Attributes describe a span with bounded, queryable data; events mark moments inside it; links connect causally related traces such as batch consumers.

```
span.set_attribute('http.route', '/orders/{id}')
span.add_event('cache_miss')
```

```
Link(producer_span_context)
```

Interview tip: Use semantic convention names.

📚 Official docs

- OpenTelemetry: Signals (traces, metrics, logs)

certs.eknathalabs.com

Q44 How do you read a trace waterfall to find a latency problem?

Tracing

🎙 Spoken answer · ~61 s

In the waterfall each span is a bar positioned in time, with children nested under parents. I first find the critical path: the chain of spans that determines the total duration. A long span with little child activity may be the actual work in that service, or it may be waiting. Gaps between a parent's start and its first child, or between children, point to queueing, lock waits, thread pool starvation or garbage collection pauses. A staircase of sequential calls that do not depend on each other suggests an opportunity for parallelism. Many small repeated spans to the same database indicate an N plus one query pattern. Spans with retries or timeouts show dependency instability. Comparing a slow trace with a fast one for the same route highlights the differing step. Then I use the span attributes and linked logs to confirm the cause before changing anything.

Likely follow-ups

- What does a gap in the waterfall indicate?
- How do you spot an N+1 pattern?

Look for the longest spans, gaps with no child work, serial calls that could be parallel, repeated calls and retries.

```
traceID=<id> # open in Tempo/Jaeger
```

```
sum by (span_name) (rate(traces_spanmetrics_duration_seconds_sum[5m]))
```

Interview tip: Gaps often mean waiting, not working.

📚 Start here (official docs)

- Prometheus documentation

certs.eknathalabs.com

Q45 How do you derive RED metrics from traces, and what are span metrics?

Tracing

🎙 Spoken answer · ~57 s

Span metrics convert tracing data into metrics: the OpenTelemetry collector's spanmetrics connector, or backend features in Tempo and others, count spans and measure durations per service, operation and status, producing request rate, error rate and latency histograms. This gives RED metrics for every instrumented service with no separate metric instrumentation, and ties metrics to the same naming as traces. The critical point is to generate them before sampling, otherwise counts would reflect the sample rather than real traffic. I keep dimensions bounded, using service, route template and status code, never raw identifiers, to avoid cardinality blowups. These metrics can power dashboards and SLO alerts, and exemplars on the histograms link a latency spike to representative traces. It is a good way to get consistent baseline monitoring across heterogeneous services, though application-specific business metrics still need explicit instrumentation.

Likely follow-ups

- Why generate metrics before sampling?
- What dimensions should be used?

A collector connector aggregates spans into request count, error count and duration histograms, giving metrics without extra instrumentation.

```
connectors:
  spanmetrics:
    histogram:
      explicit:
        buckets: [5ms,25ms,100ms,500ms,2s]
```

Interview tip: Generate before sampling to keep counts exact.

📚 Official docs

- OpenTelemetry: Signals (traces, metrics, logs)

certs.eknathalabs.com

Q46 What is multi-window multi-burn-rate alerting?

Alerting

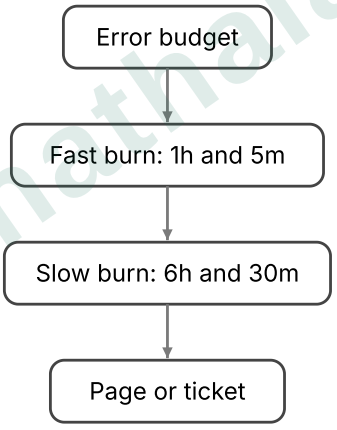
🎙 Spoken answer · ~62 s

Burn rate is how fast the error budget is being consumed relative to the rate that would exactly exhaust it at the end of the SLO window. A burn rate of one uses the budget in precisely the full period; fourteen point four over one hour consumes two percent of a thirty-day budget. A multi-window approach pairs a long window, which provides significance, with a short window, which shows the problem is ongoing and lets the alert reset quickly after recovery. Typically I use several tiers: a high burn rate over one hour and five minutes pages immediately for severe outages; a moderate burn over six hours and thirty minutes pages for slower serious problems; and a low burn over days creates a ticket. This design catches both sudden outages and slow degradations with few false alarms, and ties urgency directly to the user-facing objective and the remaining budget.

Likely follow-ups

- What does a burn rate of 1 mean?
- Why combine two windows?

🗺 Multi-window burn rate



Alert when the error budget burns quickly over both a long and short window, catching fast and slow burns while avoiding flapping.

```
(sum(rate(errors[1h]))/sum(rate(total[1h])) > 14.4*0.001) and
(sum(rate(errors[5m]))/sum(rate(total[5m])) > 14.4*0.001)
```

Interview tip: Short window confirms it is still happening.

📚 Official docs

- Google SRE Workbook: Alerting on SLOs
- Prometheus: Alerting rules

certs.eknathalabs.com

Q47 How do you write alert annotations and runbook links that help on-call responders?

Alerting

🎙 **Spoken answer · ~60 s**

An alert is a message to a tired human, so it must carry enough context to act. The title states what is wrong in plain language, not an internal metric name. The description gives the scope and impact, for example which service, which region, the current value and threshold, and how long it has lasted. Labels set severity and the owning team for routing. Links go to the relevant dashboard filtered to the right time and service, to the runbook with diagnostic steps and known remediations, and perhaps to recent deployments. A runbook should start with quick checks and decision points, include commands, escalation contacts and rollback steps, and be tested and updated after incidents. I review alerts regularly: any alert that did not require action should be fixed or removed, and any incident without a good alert reveals a gap. Templates enforce the standard.

Likely follow-ups

- What should every alert include?
- How do you keep runbooks current?

Include summary, impact, severity, the dashboard and runbook URL, and likely first checks so the responder can act in minutes.

```
annotations:
  summary: 'Checkout error rate above SLO burn threshold'
  runbook_url: https://runbooks.example/checkout-errors
  dashboard: https://grafana.example/d/checkout
```

Interview tip: The alert should answer: what, how bad, what now.

📚 **Official docs**

- Google SRE Workbook: On-Call

certs.eknathalabs.com

Q48 How do you reduce alert noise with grouping, inhibition and deduplication?

Alerting

🎙 Spoken answer · ~62 s

During a failure, dozens of alerts may fire from one root cause: a node down triggers pod, service and latency alerts. Alertmanager helps by grouping alerts by labels like cluster and alert name into a single notification, with a short group wait to collect related ones. Inhibition rules suppress lower-level alerts when a higher-level one is firing, such as hiding pod readiness alerts for a node already reported down. Deduplication ensures that repeated evaluations of the same alert do not create new pages, and repeat intervals control reminders. Routing sends alerts to the right team and channel by severity, and time-based muting suppresses expected noise during maintenance windows through silences. Beyond tooling, the best noise reduction is better alert design: alert on symptoms rather than causes, set sensible durations, and delete alerts nobody acts on. I track pages per shift as a health metric for the on-call rotation.

Likely follow-ups

- What does inhibition do?
- Why group by cluster?

Group related alerts into one notification, inhibit symptom alerts when a root-cause alert fires, and deduplicate repeated firings.

```
route:
  group_by: [alertname, cluster]
  group_wait: 30s
inhibit_rules:
- source_matchers: [alertname="NodeDown"]
  target_matchers: [alertname="PodNotReady"]
  equal: [node]
```

Interview tip: One incident should be one page.

📚 Official docs

- Google SRE Book: Practical Alerting
- Prometheus: Alertmanager

certs.eknathalabs.com

Q49 What is the difference between alerting on symptoms versus causes, and which should page?

Alerting

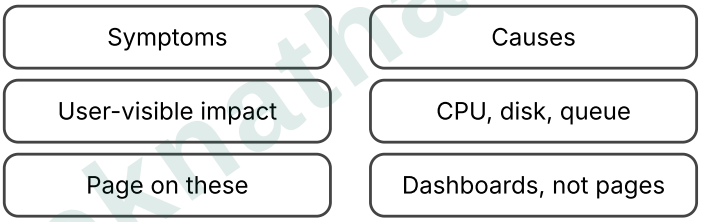
🎙 Spoken answer · ~60 s

A symptom alert reports that users are being harmed or soon will be, such as elevated error rate, high latency or failed checkouts. A cause alert reports an internal condition like high CPU, a restart, a full queue or a lagging replica, which may or may not lead to user impact. Paging on causes generates many false alarms because systems often tolerate high CPU or restarts without harm, and it also misses failures whose cause I did not anticipate. Paging on symptoms catches any cause. Cause-oriented alerts still have a place as warnings or tickets, for predictable resource exhaustion that needs action before it becomes user-visible, such as disk filling in a day, and as diagnostic panels on dashboards. The resulting structure is a small number of high-signal pages tied to service objectives, supported by richer diagnostic data to find the cause quickly.

Likely follow-ups

- Why do cause alerts cause false pages?
- Where do predictive disk alerts belong?

🗺 What to alert on



Symptoms are user-visible failures and should page; causes are internal conditions best used for diagnosis or tickets.

```
# page: error rate and latency SLO burn
# ticket: disk 80% full, node memory high
```

Interview tip: Page on what users feel.

📚 Start here (official docs)

- Prometheus documentation

certs.eknathalabs.com

Q50 How do you test alerts and make sure they actually fire?

Alerting

🎙 Spoken answer · ~60 s

Alerting rules are code and need tests. With promtool I write unit tests that feed synthetic time series into rules and assert which alerts fire at specific times, which catches typos and logic mistakes in expressions and for durations. To validate the whole chain, I use a watchdog alert that always fires and is routed to an external dead man's switch service, which notifies me if it stops arriving, meaning Prometheus, Alertmanager or the network path to the pager is broken. I also run controlled experiments, such as injecting errors or latency in staging, or in production game days, and confirm that the right alert fires, reaches the right person and has a useful runbook. After each incident I check whether the alerts fired as expected and how long detection took, and fix gaps. Regularly auditing rules for missing metrics prevents silent failures.

Likely follow-ups

- What does a dead man's switch detect?
- How does promtool test rules?

Unit-test rules with promtool, inject synthetic failures, use a dead man's switch, and run game days to validate the full path.

```
promtool test rules tests.yml
```

```
vector(1) # always-firing Watchdog alert
```

Interview tip: An alert never tested is a hope.

📚 Start here (official docs)

- Prometheus documentation

certs.eknathalabs.com

Q51 How do you choose good SLIs for different types of services?

SLOs

🎙 Spoken answer · ~60 s

A service-level indicator is a measurable proportion of good events over valid events. For request-driven services the common SLIs are availability, the fraction of requests that succeed, and latency, the fraction faster than a threshold, measured at the load balancer or client edge so they reflect the user's experience. For batch and data pipelines, freshness, meaning how recently the data was updated, correctness and throughput are better. For storage systems, durability and availability of reads and writes matter. Critical user journeys, like login or checkout, deserve their own SLIs, and I validate the indicator against real incidents: if users were unhappy while the SLI was green, it is wrong. Thresholds should come from user expectations and what is achievable, not arbitrary. I keep the number of SLIs per service small, exclude invalid events such as bad client requests, and define them in a shared template.

Likely follow-ups

- What is a good-over-valid ratio?
- How do you validate an SLI?

Pick indicators users feel: availability and latency for request services, freshness and correctness for pipelines, durability for storage.

```
sum(rate(http_requests_total{code!~"5.."}[5m])) / sum(rate(http_requests_total[5m]))
```

Interview tip: Measure close to the user.

📚 Official docs

- Google SRE Book: Service Level Objectives
- Google SRE Workbook: Implementing SLOs

certs.eknathalabs.com

Q52 How do you set an SLO target and negotiate it with product owners?

SLOs

🎙 Spoken answer · ~60 s

I start with data: what has the service actually achieved over the last months, and what do users need? A target should be tight enough that violations mean unhappy users but loose enough to be achievable and leave room for change, since each additional nine costs exponentially more and slows feature delivery. With product owners I convert targets into concrete terms: 99.9 percent over thirty days allows about 43 minutes of downtime, and ask whether that meets expectations. The negotiation also covers the error budget policy: what happens when the budget is nearly spent, for example pausing risky launches and focusing on reliability work, and what happens when there is plenty left, such as faster releases and experiments. I document it, review it quarterly, and adjust as the product matures. Internal SLOs should be stricter than any external SLA to give a buffer.

Likely follow-ups

- Why is 100% a bad target?
- What is an error budget policy?

Base it on user needs and historical performance, make it slightly below what users would notice, and agree what happens when the budget is spent.

```
# 99.9% over 30 days = 43.2 minutes of error budget
```

Interview tip: 100% is the wrong target.

📚 Official docs

- Google SRE Book: Service Level Objectives
- Google SRE Workbook: Implementing SLOs

certs.eknathalabs.com

Q53 How do you handle SLOs for dependencies and composite services?

SLOs

🎙 Spoken answer · ~60 s

If a request needs three services in series, each at 99.9 percent, the combined availability is roughly 99.7 percent, so a service cannot promise more than its critical dependencies provide. This arithmetic drives design. I identify which dependencies are hard, whose failure fails the request, and which are soft and can degrade gracefully. For hard dependencies I require SLOs higher than my own target and track their actual performance, and I reduce coupling with caching, fallbacks, timeouts, retries with budgets, circuit breakers and asynchronous processing. Redundancy across zones or providers improves effective availability for dependencies I control. When I report my SLI, I measure from the user's perspective so dependency failures count, but I also attribute budget burn to causes, which tells where investment pays off. Agreements with other teams, such as internal SLAs, make expectations explicit and provide a basis for prioritising reliability work.

Likely follow-ups

- How does serial availability compound?
- What is a soft dependency?

Availability multiplies across serial dependencies; set dependency SLOs higher than yours and use redundancy, fallbacks and caching to decouple.

```
# 0.999 * 0.999 * 0.999 ≈ 0.997
```

Interview tip: Serial dependencies erode availability.

📚 Official docs

- Google SRE Book: Service Level Objectives

certs.eknathalabs.com

Q54 How do you report SLO status to leadership and use it in planning?

SLOs

🎙 Spoken answer · ~60 s

Leaders need a clear, regular view, not a wall of graphs. For each critical service I show the SLO target, the current compliance over the window, the percentage of error budget remaining and the recent burn trend, with a status indicator. A monthly or quarterly reliability review looks at services that missed or are close to missing their objectives, the incidents that consumed budget, and the actions taken. This connects reliability to planning: services with exhausted budgets get reliability work prioritised, in line with the agreed policy, while services with healthy budgets can ship faster. I highlight the largest causes of budget loss, such as a recurring dependency failure, so investments target the biggest wins, and I translate technical numbers into user and business impact where possible. Consistent definitions across teams allow comparison, and honest reporting, including misses, builds credibility for the process.

Likely follow-ups

- What belongs on an SLO status page?
- How does budget status affect planning?

Show compliance, remaining budget and burn trends in simple dashboards and reviews, link breaches to actions, and use it to prioritise reliability work.

```
1 - (sum(increase(errors[30d])) / sum(increase(total[30d])))
```

Interview tip: Make the budget visible weekly.

📚 Official docs

- Google SRE Book: Service Level Objectives

certs.eknathalabs.com

Q55 What is the difference between SLA, SLO and SLI in practice, and what happens when an SLA is breached?

SLOs

🎙 Spoken answer · ~61 s

An SLI is a measured value, such as the proportion of requests served successfully. An SLO is the target the team sets for that SLI over a period, and it drives internal decisions through the error budget. An SLA is a contractual commitment to customers, usually less strict than the internal SLO, with defined remedies such as service credits or termination rights if it is not met. In practice, I keep the SLO stricter than the SLA so that internal alarms and corrective action occur before contractual breach. When an SLO is missed, the consequence is internal: reliability work takes priority and releases may slow. When an SLA is breached, there are financial and reputational consequences, a formal notification and calculation of credits, and a thorough postmortem for customers. The SLA's definitions of what counts as downtime, measurement windows and exclusions need careful legal and engineering review.

Likely follow-ups

- Why is the SLO stricter than the SLA?
- What is the consequence of each breach?

SLI is the measurement, SLO the internal target, SLA a contract with consequences like credits; keep the SLO stricter than the SLA.

```
# SLI: 99.95% good requests; SLO: 99.9%; SLA: 99.5% with credits
```

Interview tip: SLA breaches cost money, SLO breaches cost focus.

📚 Official docs

- Google SRE Book: Service Level Objectives

certs.eknathalabs.com

Q56 How do you structure dashboards: overview, service and deep-dive levels? Dashboards

🎙 Spoken answer · ~59 s

During an incident people need to go from what is wrong to why in minutes. I create a top-level overview showing the health of all key services and SLO status, with colour coding and links. Each service has a standard dashboard with the same layout: SLO and error budget at the top, then request rate, errors and latency, then dependency health, then resource saturation such as CPU, memory and connection pools, and finally recent deployments or annotations. Deep-dive dashboards cover specific technologies like Kubernetes nodes, databases or caches. Consistency matters, because responders can read any service's dashboard without learning a new layout, and generating dashboards from code or templates ensures it. Variables for cluster, namespace and service avoid duplicates, and links carry the time range and filters between levels. I remove panels that no one uses and review dashboards after incidents.

Likely follow-ups

- What goes at the top of a service dashboard?
- Why generate dashboards from code?

Layered dashboards move from health overview to service RED panels to detailed resources, with consistent layout and drill-down links.

```
# row 1: SLO status; row 2: RED; row 3: dependencies; row 4: resources
```

Interview tip: Same layout for every service.

📚 Official docs

- Grafana documentation

certs.eknathalabs.com

Q57 What are Grafana annotations and how do they help correlate changes with incidents?

Dashboards

🎙 Spoken answer · ~59 s

A large share of incidents are caused by changes, so seeing change events on the same timeline as metrics greatly speeds diagnosis. Grafana annotations overlay vertical markers on panels for events, which can be added through the API from the CI/CD pipeline at each deployment, from configuration management, feature flag changes, scaling events or manual incident markers. They can also be queried from a data source, such as a metric or log query that identifies restarts or alerts. I include tags for service, environment and version so dashboards can filter annotations relevant to them. During an investigation, a latency jump that lines up exactly with a deploy marker immediately suggests a rollback, while no nearby change suggests looking at traffic or dependencies. Annotations also help postmortems build timelines accurately. Making the pipeline emit them is a small investment with a large payoff.

Likely follow-ups

- What events should be annotated?
- How are annotations created automatically?

Annotations mark events such as deploys, config changes and incidents on graphs, showing what changed when behaviour shifted.

```
curl -X POST /api/annotations -d '{"tags":["deploy"],"text":"web v1.4.2"}'
```

Interview tip: Most incidents follow a change.

📚 Official docs

- Grafana documentation

certs.eknathalabs.com

Q58 How do you avoid misleading visualisations such as averages, misaligned axes and missing data?

Dashboards

🎙 Spoken answer · ~61 s

Averages hide the experience of the slowest users; latency should be shown with percentiles such as p50, p95 and p99, or as heatmaps of the distribution. Mean error rates over long windows can hide short severe outages, so I choose resolution suited to the question. Other pitfalls include inconsistent or missing units, truncated axes that exaggerate small changes or zero-based axes that hide important ones, stacked graphs that make individual series hard to read, and too many lines. Missing data should be visible: a gap because a target stopped reporting is different from zero, and a dashboard that shows zero errors because the metric disappeared is dangerously misleading, so I add panels for data presence and alert on absent metrics. Rates computed over very small sample sizes are noisy, so I show request counts alongside. Clear titles, descriptions, thresholds and consistent colours make a dashboard trustworthy.

Likely follow-ups

- Why are averages misleading for latency?
- How do you detect missing data?

Use percentiles or heatmaps for latency, label units, start axes sensibly, handle gaps explicitly, and show sample sizes.

```
histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
```

Interview tip: Averages hide tail latency.

📚 Start here (official docs)

- Prometheus documentation

certs.eknathalabs.com

Q59 How do you build dashboards-as-code and manage them with GitOps?

Dashboards

🎙 **Spoken answer · ~57 s**

Dashboards edited by hand in the UI drift, break silently and cannot be reviewed or reproduced. Treating them as code solves this. I define them with a library like Grafonnet or Jsonnet, the Grafana Terraform provider, or provisioned JSON files, and keep them in Git. Templates generate the standard service dashboard for each service from a list, so consistency is enforced and a change to the template updates all of them. Pull requests provide review, and CI validates the output, for example by linting the JSON or checking queries. A pipeline or the Grafana operator then deploys them to environments. UI editing is still allowed for experimentation, but changes that matter are exported back to code, and production provisioning can be set read-only to prevent drift. This approach gives history, rollback, reuse across environments and disaster recovery.

Likely follow-ups

- Why use templates for service dashboards?
- How do you prevent drift?

Define dashboards with Jsonnet, Grafonnet, Terraform or provisioning files in Git, review changes, and deploy automatically.

```
jsonnet -J vendor dashboards/service.jsonnet > service.json
```

```
terraform apply # grafana_dashboard resources
```

Interview tip: Stop editing production dashboards by hand.

📚 **Official docs**

- OpenGitOps principles
- Grafana documentation

certs.eknathalabs.com

Q60 How would you build a dashboard for a business-critical user journey, such as checkout? Dashboards

🎙 Spoken answer · ~57 s

I begin from the journey: cart, address, payment, confirmation. For each step I show success rate and latency, and a funnel of conversion between steps, which reveals drop-offs that technical metrics alone might miss. Next to it, I show the technical dependencies supporting the journey: payment provider latency and errors, inventory service, database and cache health, along with SLO burn for the journey. Synthetic probes that perform a test purchase continuously give an outside-in measure and detect problems even at low traffic. Deploy and feature flag annotations help correlation. A business metric, such as orders per minute compared to the same time last week, shows impact in terms everyone understands. Alerts for the journey are linked from the dashboard. A shared dashboard used by engineering, support and product creates a common picture during incidents and reduces back-and-forth.

Likely follow-ups

- Why include a synthetic probe?
- Why add a week-over-week comparison?

Combine funnel conversion, step latency and errors, dependency health and synthetic checks so engineering and business see the same story.

```
sum(rate(checkout_step_total{step="payment",result="ok"}[5m])) /
sum(rate(checkout_step_total{step="cart"}[5m]))
```

Interview tip: Include a business metric next to technical ones.

📚 Official docs

- Grafana documentation

certs.eknathalabs.com

Q61 How do Thanos, Cortex and Mimir extend Prometheus for long-term storage and global views? Tools

🎙 **Spoken answer · ~59 s**

A single Prometheus keeps data locally for a limited time and is not horizontally scalable. Thanos adds a sidecar that uploads blocks to object storage, a store gateway to read them, a querier that merges data from many Prometheus instances and storage into one global view, and a compactor that downsamples for cheaper long-range queries. Cortex and Mimir take a different approach: Prometheus servers remote write to a horizontally scalable, multi-tenant backend with distributors, ingesters and queriers, storing blocks in object storage, which suits large central platforms and SaaS-like setups. Both give long retention, high availability through replication or deduplication, and global queries across clusters. I keep local Prometheus instances for scraping and alerting so alerts do not depend on the central system, and use the long-term system for dashboards, history and capacity planning. Cost control comes from cardinality management and downsampling.

Likely follow-ups

- What does the Thanos compactor do?
- Why keep alerting local?

They accept remote-written or sidecar-uploaded data, store it in object storage, and provide global query, downsampling and multi-tenancy.

```
remote_write:
- url: https://mimir.example/api/v1/push
```

```
thanos query --store=sidecar:10901
```

Interview tip: Keep local Prometheus for alerting.

📚 **Official docs**

- Prometheus: Storage (TSDB)
- Prometheus: Overview
- Thanos documentation

certs.eknathalabs.com

Q62 What is the OpenTelemetry Collector and why use it?

Tools

🎙 Spoken answer · ~58 s

The Collector is a standalone service that receives telemetry in many formats, processes it, and exports to one or more backends. Receivers accept OTLP, Prometheus scrapes, Jaeger, Zipkin and logs. Processors batch data for efficiency, limit memory, add Kubernetes metadata such as pod and namespace, redact sensitive attributes, filter noise and perform tail sampling. Exporters send data to the chosen backends, and can fan out to multiple destinations. Using a collector decouples applications from vendors: apps send OTLP to a local endpoint, and changing backends becomes a configuration change. A typical deployment runs a daemonset agent on each node for local collection and enrichment, and a gateway deployment for heavier processing such as tail sampling and routing. Production considerations are sizing, memory limiters, queues and retries for backpressure, health monitoring of the collector itself, and managing configuration through version control.

Likely follow-ups

- What are receivers, processors and exporters?
- Why use agent and gateway tiers?

A vendor-neutral pipeline of receivers, processors and exporters that ingests, enriches, samples and routes telemetry.

```
receivers: [otlp]
processors: [batch, memory_limiter, k8sattributes]
exporters: [otlphttp]
```

Interview tip: Run agents per node and a gateway tier.

📚 Official docs

- OpenTelemetry documentation

certs.eknathalabs.com

Q63 How do you choose between self-hosted open-source observability and a commercial SaaS platform?

Tools

🎙 Spoken answer · ~60 s

Self-hosted tools like Prometheus, Loki, Tempo and Grafana offer control, flexibility, no per-host licensing and data kept in my environment, but need engineers to run, scale, secure and upgrade them, and incident tooling can be fragmented. Commercial platforms provide integrated metrics, logs, traces, alerting and analysis out of the box, fast onboarding, support and advanced features, but pricing based on hosts, ingestion or custom metrics can grow steeply and surprise teams, and there is a risk of lock-in. My decision criteria include total cost at expected scale including engineer time, compliance and data residency requirements, retention and cardinality needs, team capacity and skills, and desired features. A pragmatic path is to instrument with OpenTelemetry and open metric formats so backends can change, and potentially mix: for example managed Prometheus for metrics and a SaaS for traces. Budgets and cost governance are needed in either case.

Likely follow-ups

- Why instrument with OpenTelemetry?
- What drives SaaS cost?

Weigh cost at scale, operational effort, features, data control, lock-in and team skills; many use a hybrid with OpenTelemetry for portability.

```
# criteria: cost model, retention, cardinality limits, compliance, support, exit plan
```

Interview tip: Instrument with open standards either way.

📚 Start here (official docs)

- Prometheus documentation

certs.eknathalabs.com

Q64 What is eBPF-based observability and what are its benefits and limits?

Tools

🎙 **Spoken answer · ~60 s**

eBPF lets small, verified programs run safely inside the Linux kernel, attached to events like system calls, network packets and function entry points. Observability tools use it to collect telemetry without modifying or restarting applications: network flows and DNS visibility between services, latency of TCP connections, CPU profiles, file and process activity, and even automatic HTTP or database request tracing by parsing traffic. Examples include Cilium's Hubble, Pixie, Parca and Falco for security. Benefits are low overhead, broad coverage including legacy or third-party software, and visibility from the kernel's perspective that application instrumentation can miss. Limits include the need for fairly recent kernels and elevated privileges, difficulty decoding encrypted traffic without hooks into libraries, less business-level context than application traces, operational risk if poorly configured, and cost of the data volume. I treat it as a complement to OpenTelemetry instrumentation, not a replacement.

Likely follow-ups

- What does eBPF let you observe?
- What are its limits?

eBPF programs run in the kernel to collect network, process and security telemetry with low overhead and no code changes.

```
bpftrace -e 'tracepoint:syscalls:sys_enter_openat { @[comm] = count(); }'
```

```
cilium hubble observe
```

Interview tip: Needs kernel support and privileges.

📚 **Official docs**

- Linux kernel docs: BPF
- ebpf.io: What is eBPF?

certs.eknathalabs.com

Q65 How do synthetic monitoring and real user monitoring differ and when do you need each?

Tools

🎙 **Spoken answer · ~60 s**

Synthetic monitoring runs scripted requests or browser journeys from chosen locations at regular intervals. It gives consistent baselines, detects outages even when there is no user traffic, tests critical flows end to end such as login and checkout, and measures from outside the infrastructure, so it also catches DNS, CDN or certificate issues. Its limitation is that it covers only the scenarios I script, from a few locations. Real user monitoring instruments the actual browser or mobile app, collecting page load timings, web vitals, JavaScript errors and network failures from real devices, networks and regions, showing what users truly experience, including problems specific to a browser version or country. It depends on traffic and has privacy considerations. I use synthetic checks for availability alerts and SLOs on key journeys, and RUM for performance analysis and prioritisation. Correlating both with backend traces gives the full picture.

Likely follow-ups

- What can synthetic monitoring catch that RUM cannot?
- What privacy issues does RUM have?

Synthetic checks run scripted probes continuously from outside; RUM measures actual user sessions in browsers. Together they show availability and real experience.

```
blackbox_exporter probe module: http_2xx target: https://shop.example
```

```
probe_success{job="blackbox"} == 0
```

Interview tip: Synthetics find outages at 3 a.m. with no traffic.

📚 **Official docs**

- Google SRE Book: Distributed tracing & logs (Monitoring chapter)
- Prometheus Blackbox exporter

certs.eknathalabs.com

Q66 How do you monitor Kubernetes control-plane health: API server, etcd and scheduler?

K8s obs

🎙 Spoken answer · ~61 s

On self-managed clusters the control plane is critical, and even on managed ones some metrics are exposed. For the API server I watch request rate, error rate and latency by verb and resource, inflight requests and priority and fairness rejections. For etcd, key signals are write-ahead log fsync and backend commit durations, which reflect disk speed, leader changes, proposals failed or pending, database size against quota and the number of members up. The scheduler's pending pod count and scheduling latency show capacity or constraint issues; the controller manager's work queue depth and retries show reconciliation problems. Certificate expiry for control-plane components is another classic outage cause. I alert when the API server error rate or latency is high, etcd has no leader or frequent leader elections, or database size nears the quota. Slow admission webhooks are a hidden cause of API latency, so I monitor them too.

Likely follow-ups

- Why is etcd disk latency crucial?
- What can slow the API server besides load?

Track API request latency and errors, etcd disk sync and leader changes, scheduler queue and pending pods, and controller work queues.

```
histogram_quantile(0.99, sum by (le,verb)
(rate(apiserver_request_duration_seconds_bucket[5m])))
```

```
etcd_disk_wal_fsync_duration_seconds_bucket
```

Interview tip: etcd is sensitive to disk latency.

📚 Official docs

- Kubernetes: Operating etcd clusters
- etcd documentation

certs.eknathalabs.com

Q67 How do you use kube-state-metrics and cAdvisor, and what is the difference? K8s obs

🎙 Spoken answer · ~59 s

They answer different questions. Kube-state-metrics listens to the Kubernetes API and generates metrics about the desired and current state of objects: deployments' available replicas, pod phase, container waiting reasons like CrashLoopBackOff, job success, node conditions, resource requests and limits, persistent volume claim status. cAdvisor, built into the kubelet, measures actual resource consumption of each container: CPU time, memory working set, network and disk I/O, and CPU throttling. Combining the two is powerful: comparing usage with requests and limits shows over- and under-provisioning, and state metrics tell me why a pod is not running while usage metrics tell me how it behaves when running. Typical alerts use state metrics such as unavailable replicas or pods stuck pending, while capacity planning and right-sizing use usage metrics. Both are deployed by kube-prometheus-stack and need label joins to attach owner information such as the deployment name.

Likely follow-ups

- Which tool shows CrashLoopBackOff?
- How do you right-size from these metrics?

kube-state-metrics reports object state from the API; cAdvisor reports container resource usage from the kubelet.

```
kube_deployment_status_replicas_unavailable > 0
```

```
rate(container_cpu_usage_seconds_total{container!=""}[5m])
```

Interview tip: State vs usage.

📚 Official docs

- Kubernetes: Metrics for system components
- kube-state-metrics

certs.eknathalabs.com

Q68 How do you find noisy neighbours and right-size workloads using metrics?

K8s obs

🎙 **Spoken answer · ~60 s**

I compute, for each workload, the ratio of actual usage to the requested amount using percentiles over a long enough period to include peaks, such as the ninety-fifth percentile over two weeks. Workloads using far less than requested waste money and block scheduling; those near or above limits risk throttling or OOM kills. I include CPU throttling metrics and restart reasons to catch workloads constrained in ways usage hides. For noisy neighbours, I look at nodes with high CPU steal, memory pressure or disk and network saturation and identify the top consumers by node. The vertical pod autoscaler in recommendation mode analyses history and suggests requests, which I apply gradually and review rather than blindly automating. Namespace quotas and limit ranges keep teams within bounds, and cost tools allocate spend to owners. Right-sizing is a continuous process, repeated after significant changes in load or code.

Likely follow-ups

- Why use p95 over a long window?
- What does VPA recommendation mode do?

Compare usage against requests per workload over weeks, look at percentiles and throttling, and use VPA recommendations.

```
quantile_over_time(0.95, container_memory_working_set_bytes{namespace="app"}[7d])
```

```
kube_pod_container_resource_requests{resource="cpu"}
```

Interview tip: Use p95 over a long window.

📚 **Official docs**

- Kubernetes: Multi-tenancy

certs.eknathalabs.com

Q69 How do you monitor pod restarts, OOM kills and evictions meaningfully?

K8s obs

🎙 Spoken answer · ~61 s

A restart counter by itself is noisy; what matters is the rate and the reason. I alert when a container restarts several times within an hour, using increase over a window, and enrich the alert with the last termination reason from kube-state-metrics. OOMKilled points to memory limits or leaks and leads to checking memory trends and heap settings. A non-zero exit code with error reason suggests an application crash, so I look at previous logs. Evictions happen when the node is under pressure, such as memory or disk, and I correlate them with node metrics and with pods' QoS classes; many evictions on a node suggest requests are too low or the node is overcommitted. CrashLoopBackOff waiting reasons are alerted separately. Dashboards show restarts by workload to spot patterns after deploys. Each alert links to a runbook with the relevant kubectl describe, logs and metrics steps.

Likely follow-ups

- How do you tell OOM from an application crash?
- What causes evictions?

Alert on restart rate and last termination reason, not raw counts; separate OOMKilled from errors and evictions from node pressure.

```
increase(kube_pod_container_status_restarts_total[1h]) > 3
```

```
kube_pod_container_status_last_terminated_reason{reason="OOMKilled"} == 1
```

Interview tip: Reason labels tell you the fix.

📚 Official docs

- Kubernetes: Node-pressure eviction
- Linux kernel docs: /proc/sys/vm

certs.eknathalabs.com

Q70 How do you set up multi-cluster monitoring and consistent labelling?

K8s obs

🎙 **Spoken answer · ~60 s**

With many clusters, I give each Prometheus or agent external labels such as cluster, region and environment, so every series and alert carries its origin. Metrics are shipped to a central store using remote write, or queried globally with Thanos, and logs and traces are tagged with the same attributes through the collectors. Dashboards use a cluster variable, so one dashboard serves all clusters, and alert rules are written once and deployed everywhere, with the cluster label used in routing and annotations. A consistent label schema across telemetry is essential: the same names for service, namespace, team and version, following OpenTelemetry resource conventions, so I can pivot between metrics, logs and traces. Ownership labels enable routing alerts to the right team. I also monitor the monitoring: each cluster's agent health, scrape failures and remote write lag, with a watchdog alert so silent failures are noticed.

Likely follow-ups

- Why use external labels?
- How do you monitor the monitoring system?

Add cluster, region and environment external labels, ship to a central store, and use consistent label schemas for dashboards and alerts.

```
global:
  external_labels:
    cluster: prod-eu-1
    region: eu-west-1
```

Interview tip: Add the cluster label at the source.

📚 **Start here (official docs)**

- Prometheus documentation

certs.eknathalabs.com

Q71 How do you monitor serverless functions such as Lambda or Cloud Functions?

Cloud obs

🎙 Spoken answer · ~59 s

Serverless removes servers to manage but not the need for observability. The platform provides core metrics: invocation count, errors, duration percentiles, throttles when concurrency limits are hit, and iterator age for stream triggers. I alert on error rate, throttling, and duration approaching the timeout. Cold starts add latency to a fraction of invocations, visible in tail latency and in init duration metrics, and mitigated with provisioned concurrency, smaller packages or lighter runtimes. Logs are written to the platform's logging service; I use structured JSON with request identifiers and keep log volume reasonable, as it affects cost. Distributed tracing, through OpenTelemetry layers or the provider's tracing integration, links functions with the services they call and with queues or API gateways. I also monitor downstream limits like database connections that functions can exhaust when scaling quickly, and track cost per invocation as a signal.

Likely follow-ups

- What causes throttles in Lambda?
- How do cold starts show up in metrics?

Track invocations, errors, duration, throttles and cold starts, use structured logs and distributed tracing, and alert on error rate and concurrency limits.

```
aws cloudwatch get-metric-statistics --namespace AWS/Lambda --metric-name Errors ...
```

```
gcloud functions logs read my-func
```

Interview tip: Cold starts show up in tail latency.

📚 Official docs

- AWS Lambda Developer Guide

certs.eknathalabs.com

Q72 How do you control observability costs in the cloud?

Cloud obs

🎙 Spoken answer · ~60 s

Telemetry costs grow quickly with scale. I attack it from several angles. For metrics, limit label cardinality, drop unused metrics at scrape time and set longer scrape intervals for low-value targets. For logs, reduce verbosity, sample routine events, drop health-check noise, filter before ingestion, and use cheaper storage tiers or archives for older logs with short searchable retention. For traces, apply tail sampling to keep errors and slow requests and a small fraction of the rest. I remove duplicates, such as the same logs collected twice, and unused dashboards and queries. Visibility drives behaviour: I track ingestion volume and cost by team and service, set budgets and alerts, and make owners accountable. Retention should reflect need: seven days of detailed data is often enough for debugging, with aggregates kept for longer. Costs must be balanced against the risk of losing data needed during an incident.

Likely follow-ups

- Where do telemetry costs come from?
- How do you make teams accountable for them?

Manage cardinality, sampling, retention tiers, log volume and duplicate collection, with budgets, ownership and dashboards of telemetry spend.

```
topk(10, sum by (job) (scrape_samples_scraped))
```

```
aws logs describe-log-groups --query 'logGroups[].[logGroupName,storedBytes]'
```

Interview tip: Observability is often the second-largest bill.

📚 Official docs

- OpenTelemetry: Logs

certs.eknathalabs.com

Q73 How do you build a cross-account or cross-project logging and monitoring architecture?

Cloud obs

🎙 Spoken answer · ~58 s

In multi-account organisations, each workload account would otherwise keep its own islands of logs and metrics. A common design is a dedicated observability or logging account or project. Workload accounts send logs and metrics through sinks, subscription filters or agents to central storage, with write-only permissions so a compromised workload cannot alter or delete the central copy. Organisation-level sinks automatically capture audit logs from new accounts. The central platform provides search, dashboards, alerting and long retention, with role-based access so teams see their own data and security teams see audit streams. Data is encrypted with managed keys, transfer costs are considered, and residency rules may require regional aggregation. Metrics may stay local for alerting while summaries flow centrally. Infrastructure as code deploys the agents, sinks and permissions consistently so that a new account is observable from day one.

Likely follow-ups

- Why write-only access from workloads?
- What is an organisation-level sink?

Centralise into a dedicated observability account or project with delegated, read-only access, aggregation sinks and strict IAM and encryption.

```
gcloud logging sinks create central-sink storage.googleapis.com/bucket --include-children --organization=ORG
```

```
aws logs put-subscription-filter ...
```

Interview tip: Write-only from workloads, read-controlled centrally.

📚 Start here (official docs)

- Prometheus documentation

certs.eknathalabs.com

Q74 How do you use cloud-native tracing and profiling such as X-Ray, Cloud Trace and Cloud Profiler?

Cloud obs

🎙 Spoken answer · ~61 s

Cloud providers offer tracing services that integrate with their managed products such as load balancers, functions, queues and databases, giving traces that include those services without much manual work, plus application SDKs or OpenTelemetry exporters for custom code. Traces show the path and latency of requests across services and help find bottlenecks and errors, and service maps visualise dependencies. Continuous profilers like Cloud Profiler or similar tools sample stack traces from production processes with low overhead and show flame graphs over time, revealing which functions consume CPU or allocate memory, and how a release changed that. Together they answer different questions: tracing explains where in the distributed path time goes, profiling explains why a service itself is slow. For portability I prefer OpenTelemetry for instrumentation with the cloud service as a backend, and I make sure sampling and retention settings meet both debugging needs and cost constraints.

Likely follow-ups

- What does continuous profiling show?
- How do traces and profiles complement each other?

Instrument or enable integrations to trace requests across managed services, and use continuous profilers to find CPU and memory hotspots in production.

```
aws xray get-trace-summaries --start-time ... --end-time ...
```

```
gcloud trace traces list
```

Interview tip: Continuous profiling has very low overhead.

📚 Official docs

- AWS X-Ray
- Cloud Trace

certs.eknathalabs.com

Q75 How do you run an incident bridge: roles, communication and decision-making?

Incident

🎙 Spoken answer · ~58 s

A clear structure prevents chaos. The incident commander owns coordination and decisions, delegating investigation instead of diving into it. A technical lead or leads drive diagnosis and mitigation in their areas, a scribe records the timeline, actions and decisions, and a communications lead updates stakeholders and customers on a regular cadence. Everyone works in one channel or call so information is shared, and side conversations are summarised back. The commander asks for current status, hypotheses and next steps, time-boxes them, and prefers reversible mitigation, like rollback or failover, over finding the root cause during the outage. Severity is declared early and revisited, and people are rotated to avoid fatigue in long incidents. When service is restored, the commander declares the incident mitigated, assigns follow-ups, and schedules a blameless postmortem. Practising with game days makes roles familiar before a real emergency.

Likely follow-ups

- Why should the commander not debug?
- Why prefer mitigation over root cause during an outage?

Assign an incident commander, scribe, communications lead and technical leads; keep a single channel, regular updates and time-boxed hypotheses.

```
/incident declare sev2 'checkout errors'
```

```
# update every 15-30 min
```

Interview tip: The commander coordinates, not debugs.

📚 Official docs

- Google SRE Book: Managing Incidents

certs.eknathalabs.com

Q76 How do you quickly determine the blast radius and customer impact of an incident?

Incident

🎙 Spoken answer · ~58 s

First I look at the symptom metrics sliced by available dimensions: region, availability zone, cluster, service version, endpoint, customer tier and tenant. If errors are concentrated in one zone or one new version, that both limits the blast radius and suggests the cause and mitigation. I compare the proportion of affected requests to the total, translating it into the number or share of users, and check how quickly the error budget is burning. External signals complement internal ones: support ticket volume, social media mentions, synthetic probes and real user monitoring. For multi-tenant systems, I identify whether specific tenants or a shared component is the problem. This scoping drives severity, communication content and who needs to be involved. I update the scope as I learn more, and record it in the timeline, because stakeholders need an honest, evolving estimate of impact.

Likely follow-ups

- Why slice by version and zone?
- Which external signals help?

Segment errors by region, tenant, version and endpoint, check SLO burn, support tickets and synthetic checks to scope who is affected.

```
sum by (region, version) (rate(http_requests_total{code=~"5.."}[5m]))
```

```
sum by (tenant) (rate(errors_total[5m]))
```

Interview tip: Slice by every dimension you have.

📚 Official docs

- AWS: Disaster recovery & availability concepts (Fault isolation boundaries)
- AWS Builders' Library: Workload isolation using shuffle sharding

certs.eknathalabs.com

Q77 What makes a postmortem action item effective, and how do you track them?

Incident

🎙 Spoken answer · ~58 s

Weak action items say things like be more careful or improve monitoring. Effective ones are concrete, for example add an alert when queue age exceeds five minutes, with an owner, a due date and a link to the ticket. I prioritise by how much risk each removes, and favour systemic fixes such as automation, guardrails, safer defaults or removal of single points of failure over relying on humans to remember. Items are classified: detect faster, mitigate faster, prevent recurrence. They go into the same backlog and planning process as feature work, with agreed capacity, otherwise they get ignored. I review open actions regularly, report completion rates to leadership, and revisit if an action is blocked or no longer worthwhile, recording the decision. Checking whether similar incidents recur and whether detection and recovery times improved shows whether the actions worked.

Likely follow-ups

- What makes a weak action item?
- How do you ensure they get done?

Specific, owned, time-bound, prioritised by risk reduction, and tracked in the normal backlog with follow-up reviews.

```
# action: add alert on queue age > 5m; owner: team-a; due: 2026-11-15; priority: P1
```

Interview tip: Prefer systemic fixes over reminders.

📚 Official docs

- Google SRE Book: Postmortem Culture
- Google SRE Workbook: Postmortem Culture
- Atlassian: How to run a blameless postmortem

certs.eknathalabs.com

Q78 How do you handle alert storms during a major outage?

Incident

🎙 Spoken answer · ~58 s

In a large outage hundreds of alerts can fire, burying the signal and exhausting responders. During the incident, the commander designates one person to triage alerts, group them by common cause, and identify the root-cause alert, such as a node pool failure or a database outage. Silencing derivative alerts for a bounded duration, with a comment and an owner, keeps pagers quiet so people can think, while the main alert and the incident channel carry the information. Inhibition rules configured in advance should do much of this automatically. I make sure silences expire and are reviewed after the incident, because a forgotten silence can hide a real problem. Afterwards, I analyse which alerts were useful, add grouping and inhibition, remove redundant ones, and improve dashboards that show dependency relationships. Alert storm handling is part of on-call training and game days.

Likely follow-ups

- What should be silenced?
- Why must silences expire?

Silence or group symptoms, focus on the root-cause signal, designate who triages alerts, and fix the alert design afterwards.

```
amtool silence add alertname=~"PodNotReady|HighLatency" cluster=prod-eu-1 --duration=1h --
comment='incident 1234'
```

Interview tip: Silences need an owner and expiry.

📚 Start here (official docs)

- Prometheus documentation

certs.eknathalabs.com

Q79 How do you measure on-call health and reduce burnout?

Incident

🎙 Spoken answer · ~59 s

On-call is part of the system and its health affects reliability. I track pages per shift, how many occur outside working hours, the proportion that were actionable, time to acknowledge and time to resolve, and escalations. Targets such as fewer than two incidents per shift are common guidance. When numbers are bad, I address causes: eliminate or tune noisy alerts, automate frequent manual remediation, improve runbooks, and fix recurring root causes with reliability work scheduled in planning. Process measures include enough people in the rotation, fair distribution including for holidays, follow-the-sun coverage where possible, secondary responders for support, handoff notes, and compensatory time off after bad nights. I also run regular on-call reviews where engineers describe pain points, and ensure new members shadow before taking the pager. Leadership should treat unsustainable on-call as a priority issue, because exhausted responders make more mistakes.

Likely follow-ups

- Which metrics show on-call is unhealthy?
- What reduces recurring pages?

Track pages per shift, out-of-hours pages, time to acknowledge and actionable ratio; fix noisy alerts, share load fairly and give recovery time.

```
# metrics: pages/shift, night pages, % actionable, MTTA, MTTR
```

Interview tip: Sustainable on-call is a reliability requirement.

📚 Official docs

- Google SRE Book: Being On-Call
- Google SRE Workbook: On-Call

certs.eknathalabs.com

Q80 How do you write an incident timeline and perform root cause analysis without blaming people? Incident

🎙 **Spoken answer · ~58 s**

I build the timeline from objective sources: monitoring data, deployment records, audit logs and the chat transcript, noting when events happened, when they were detected and when actions were taken. Then I analyse contributing factors rather than a single root cause, because complex failures usually involve several conditions: a latent bug, a missing safeguard, a misleading alert, a gap in documentation. Techniques like the five whys help, but I keep asking how the system allowed it, not who made the mistake. A blameless approach assumes people acted reasonably given what they knew, so I ask what information they had and why the action seemed sensible. This encourages honest reporting and surfaces real weaknesses. The output includes impact, timeline, detection and response assessment, contributing factors and action items, shared widely so other teams learn. Blame hides facts, while curiosity reveals them.

Likely follow-ups

- Why avoid a single root cause?
- What does blameless mean in practice?

Reconstruct events from logs and chat, ask how the system allowed the failure using techniques like five whys, and focus on contributing factors.

```
# timeline: 10:02 deploy; 10:07 errors rise; 10:11 page; 10:25 rollback; 10:31 recovery
```

Interview tip: Why did it make sense at the time?

📚 **Official docs**

- Google SRE Book: Postmortem Culture

certs.eknathalabs.com